

The University of Faisalabad

Lab Manual

Data Mining

DS-303

Submitted by

Muneeb Sajid 2023-BS-AI-028

Submitted to

Sir Arham

Department of Computer Science

Table of Contents

LAB NO.	TOPIC	PAGE NO.
LAB-1	KNIME Introduction	3
LAB-2	Customer Churn Prediction	7
LAB-3	Data Cleaning	13
LAB-4	Data Preprocessing	17
LAB-5	Market Basket Analysis	24
LAB-6	Apriori Algorithm (Product Combination Analysis)	29
LAB-7	FP-Growth (Efficient Frequent Pattern Mining)	32
LAB-8	Python Basics (Reusable Data Processing Module)	37
LAB-9	Decision Tree	41
LAB-10	Naïve Bayes & KNN	45
LAB-11	SVM	50
LAB-12	Clustering	54
LAB-13	Outlier Detection	54s
LAB-14	Scraping & Sentiment Analysis	58
LAB-15	Final Capstone	63

Lab 1:

KNIME Introduction

Real-World Scenario

A retail company stores large volumes of transaction records across different product categories. Although the data is available, it cannot support decision-making unless it is organized and examined properly. Business managers require clear analytical outputs to measure overall sales revenue, estimate average customer purchase value, identify the categories generating the greatest earnings, and determine which individual products produce the highest profit contribution. This practical task uses KNIME Analytics Platform to transform raw retail sales data into useful business intelligence through a workflow-based analytical process.

Dataset Description

The dataset applied in this exercise is a Retail Sales CSV file containing detailed transaction records. Each row represents one completed sale and includes the following fields:

- Transaction ID
- Product Title
- Product Category
- Quantity Sold
- Unit Price
- Purchase Date
- Sales Amount (or Revenue generated during processing)

The dataset is suitable for performing summarization, revenue calculations, category comparisons, and product ranking analysis.

Software Tool Used — KNIME Analytics Platform

KNIME is a visual analytics software that allows users to create data workflows by connecting functional nodes. Every node is responsible for a specific task such as importing files, cleaning data, changing data types, calculating formulas, generating summaries, or performing aggregations. When combined, these nodes build a complete data analysis pipeline that converts unstructured sales records into valuable managerial insights.

Practical Learning Outcomes

1	Load a CSV sales dataset into KNIME using the CSV Reader node.
2	Inspect and adjust column data types for accurate processing.
3	Produce descriptive statistics to understand the nature of the dataset.
4	Create a new Revenue field using the Math Formula node.
5	Analyze four major retail KPIs using GroupBy and Sorter nodes.

Procedure

1	Import the Dataset
---	--------------------

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.

Dialog - 3:1 - CSV Reader

File

Settings Transformation Advanced Settings Limit Rows Encoding Flow Variables Job Manager Selection Memory Policy

Input location

Read from: Local File System

Mode: ☒ File ☐ Files in folder

File: C:\Users\tj\Desktop\Data Mining\Lab1\retail_sales_dataset.csv

Reader options

Format

Autodetect format

Column delimiter: , Row delimiter: ☒ Line break ☐ Custom \r\n

Quote char: " Quote escape char: \"

Comment char

☒ Has column header ☐ Has RowID

☐ Support short data rows ☐ Prepend file index to RowID

Preview

The suggested column types are based on the first 10000 rows only. See 'Advanced Settings' tab.

Row ID	Transac...	Date	Custom...	Gender	Age	Product ...	Quantity	Price pe...	Total A...
Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30
Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50
Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600
Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200
Row10	11	2023-02-14	CUST011	Male	23	Clothing	2	50	100

OK Apply Cancel ?

2 Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are correctly typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.

► 1: File Table

🔗 Flow Variables

Rows: 1000

Columns: 9

Table

Statistics

☐	#	RowID	Transaction ID ↔ Number (Integer)	Date ↔ String	Customer ID ↔ String	Gender ↔ String	Age ↔ Number (Integer)	Product Category ↔ String	Quantity ↔ Number (Integer)	Price per Unit ↔ Number (Integer)	Total Amount ↔ Number (Integer)	☐
☐	1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150	
☐	2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000	
☐	3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30	
☐	4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500	
☐	5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100	
☐	6	Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30	
☐	7	Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50	
☐	8	Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100	
☐	9	Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600	
☐	10	Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200	

3 Generate Summary Statistics

Apply the Statistics node to compute descriptive measures such as mean, minimum, maximum, sum, and standard deviation for numeric attributes.

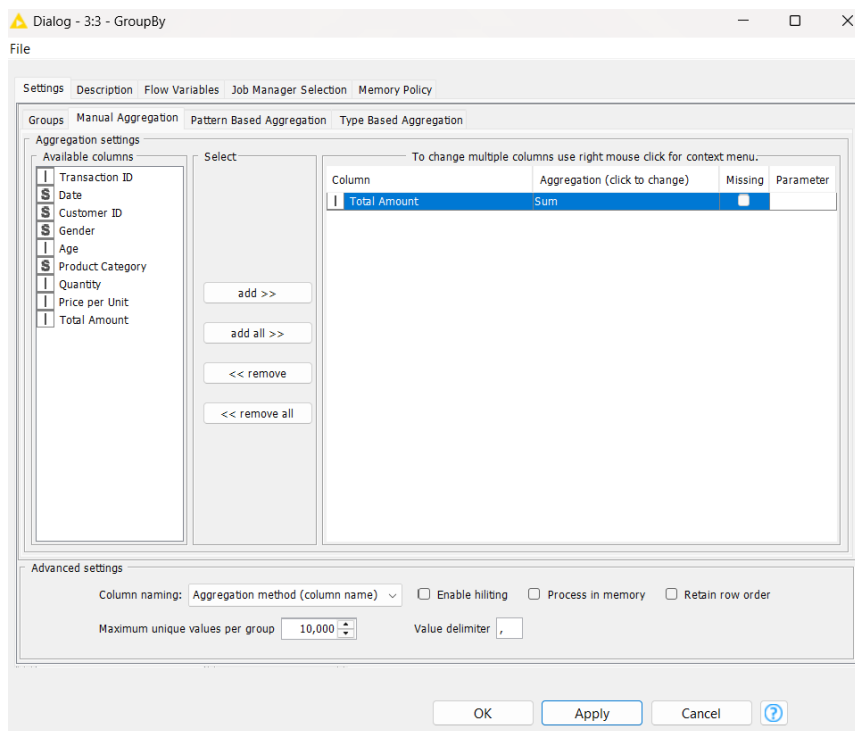
► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table  Flow Variables

Rows: 5 | Columns: 16

☐	#	RowID	Column 1: String	Min ⇒ Number (L...)	Max ⇒ Number (L...)	Mean ⇒ Number (L...)	Std. devia... ⇒ Number (L...)	Variance ⇒ Number (L...)	Skewness ⇒ Number (L...)	Kurtosis ⇒ Number (L...)	Overall su... ⇒ Number (L...)	No. missi... ⇒ Number (L...)	No. NaNs ⇒ Number (L...)	No. +cos ⇒ Number (L...)	No. -cos ⇒ Number (L...)
☐	1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0	0	0	0
☐	2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0	0	0	0
☐	3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0	0	0	0
☐	4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0	0	0	0
☐	5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0	0	0	0

4 Calculate Total Revenue (KPI 1)

Add a GroupBy node without selecting any grouping column and aggregate the Revenue column using the Sum function to compute overall sales revenue.



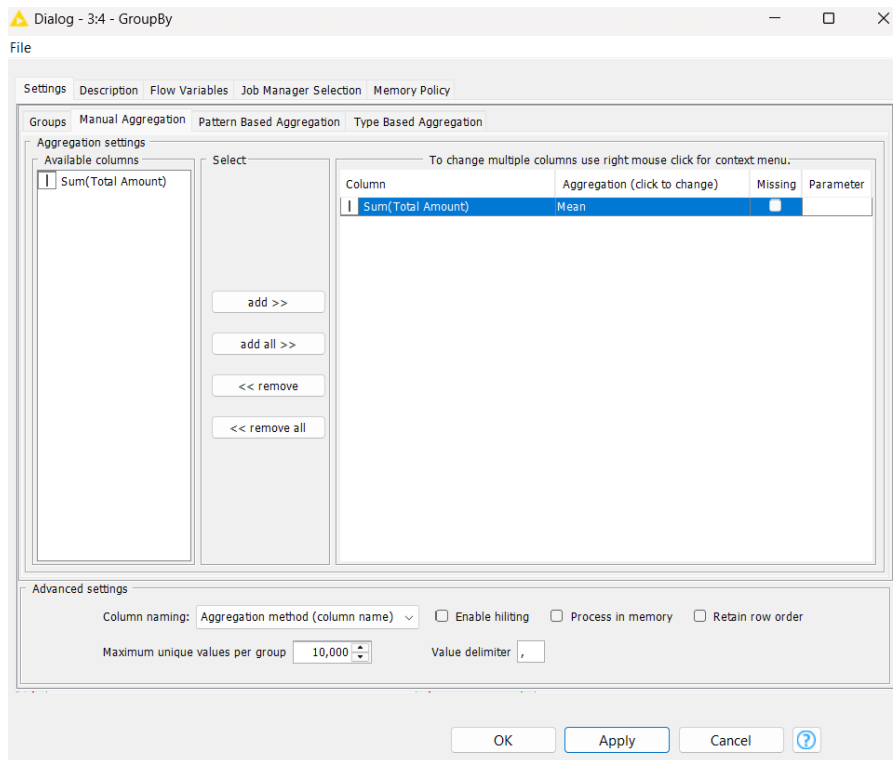
► 1: Group table  Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Sum(Total Amount) 123 Number (Integer)
☐	1	Row0	456000

5 Calculate Average Order Value (KPI 2)

Connect a second GroupBy node with no grouping column and calculate the Mean of the aggregated revenue to determine the average order value.



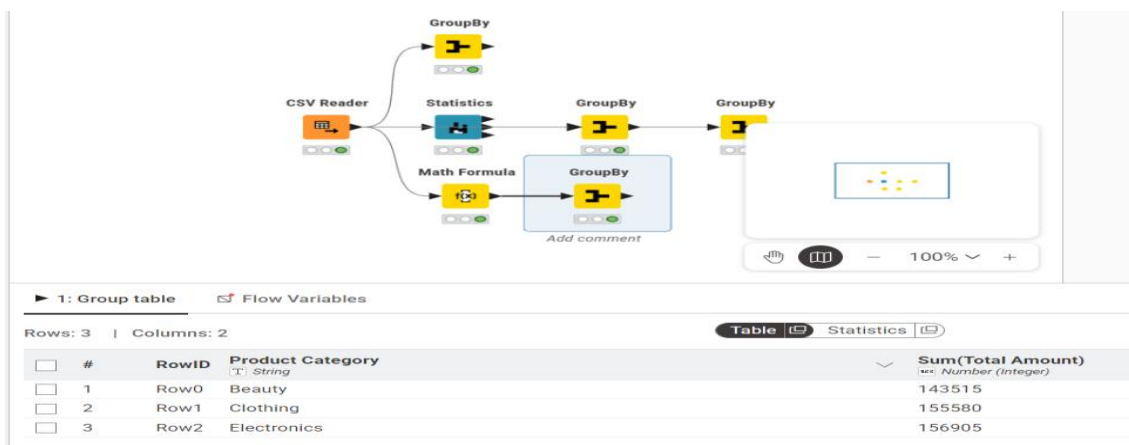
► 1: Group table Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Mean(Sum(Total Amount)) Number (Float)
☐	1	Row0	456,000

6 Calculate Category Revenue (KPI 3)

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.



Lab 2:

CRISP-DM Methodology

Topic	CRISP-DM
Real-World Problem	An e-commerce company wants to predict customer churn to improve retention
Dataset	Customer Churn Dataset
Tool(s)	Orange Data Mining
Objectives	Apply all six CRISP-DM phases, prepare data, build and evaluate predictive model

2.1 Theory and Concepts

CRISP-DM (Cross-Industry Standard Process for Data Mining) is the most widely used methodology for data mining projects. It provides a structured approach consisting of six phases:

- Business Understanding: Define project objectives and requirements
- Data Understanding: Collect data, perform initial exploration
- Data Preparation: Clean, transform, and structure data
- Modeling: Apply various modeling techniques and calibrate parameters
- Evaluation: Assess model quality against business objectives
- Deployment: Implement the model in production

2.2 Dataset Description

The Customer Churn Dataset includes customer demographics, account information, and behavioral features:

Attribute	Type	Description
Customer ID	Numeric	Unique customer identifier
Tenure	Numeric	Months as customer
Monthly Charges	Numeric	Monthly fee amount
Total Charges	Numeric	Lifetime spend
Contract Type	Nominal	Month-to-month / One year / Two year
Payment Method	Nominal	Payment type used
Churn	Binary	Yes/No - target variable

2.3 Procedure

Phase 1: Business Understanding

Define the business problem: The e-commerce company is experiencing high customer churn (estimated 25% annually). The goal is to build a predictive model that identifies at-risk customers, enabling proactive retention strategies.

Phase 2: Data Understanding

Step 1: Load the Customer Churn Dataset using the File widget in Orange.

Step 2: Use the Distributions widget to explore each feature's distribution.

Step 3: Use the Box Plot widget to identify outliers in numeric features.

Phase 3: Data Preparation

Step 4: Use Select Columns to set the target variable (Churn) and select relevant features.

Step 5: Use Preprocess to handle missing values (imputation) and normalize numeric features.

Phase 4: Modeling

Step 6: Add a Logistic Regression widget and connect the prepared data.

Step 7: Add a Random Forest widget as an alternative model for comparison.

Phase 5: Evaluation

Step 8: Connect both models to Test and Score widget using cross-validation ($k=10$).

Step 9: Add a Confusion Matrix widget to analyze prediction errors.

Step 10: Use the ROC Analysis widget to visualize model discrimination ability.

Phase 6: Deployment

Document the final model performance and create a deployment plan. The model should be retrained monthly with new customer data to maintain accuracy.

KPIs / Evaluation Metrics

KPI / Metric	Description	Target / Value
Churn Rate	Percentage of customers churning	< 20%
Retention Rate	1 - Churn Rate	> 80%
Model Accuracy	Correct predictions / Total	> 80%
ROC-AUC	Area under ROC curve	> 0.80

2.4 Real World Problem (Orange Practice)

An e-commerce company is experiencing declining revenue and suspects a significant portion of its customer base is churning. The management wants to predict which customers are likely to stop purchasing so that targeted retention campaigns can be deployed proactively. This lab applies the CRISP-DM framework to structure the entire data mining process from business understanding through to evaluation.

Dataset

Customer Churn Dataset containing customer demographics, subscription details, usage patterns, support interactions, and a binary churn label indicating whether the customer left the service.

Practical Objectives

By the end of this lab, students will be able to apply all six CRISP-DM phases to a real churn prediction problem using Orange Data Mining. Students will document business objectives, explore and prepare the dataset, build and evaluate a predictive model, and produce a deployment plan.

Procedure

Step 1. Business Understanding

Define the business goal: predict customer churn. Identify the success criteria: a model achieving at least 80% accuracy and an ROC-AUC score above 0.75. Document what churn means in the context of this company and list the data sources available.

Customer churn refers to the phenomenon where customers discontinue their relationship or subscription with a company or service provider. It represents the rate at which customers stop using a company's products or services within a specific period. Churn is an important metric for businesses as it directly impacts revenue, growth, and customer retention.

In the context of the Churn dataset, the churn label indicates whether a customer has churned or not. A churned customer is one who has decided to discontinue their subscription or usage of the company's services. On the other hand, a non-churned customer is one who continues to remain engaged and retains their relationship with the company.

Understanding customer churn is crucial for businesses to identify patterns, factors, and indicators that contribute to customer attrition. By analyzing churn behavior and its associated features, companies can develop strategies to retain existing customers, improve customer satisfaction, and reduce customer turnover. Predictive modeling techniques can also be applied to forecast and proactively address potential churn, enabling companies to take proactive measures to retain at-risk customers.

customer_churn_dataset-testing-master.csv (3.28 MB)

Detail Compact Column

10 of 12 columns

About this file

The testing file for the churn dataset consists of 64,374 customer records and serves as a separate dataset for evaluating the performance and generalization capabilities of trained churn prediction models. Each record in the testing file corresponds to a customer and contains the same set of features as the training file, such as age, gender, tenure, usage frequency, support calls, payment delay, subscription type, contract length, total spend, and last interaction. However, the churn labels are not included in the testing file as they are used for assessing the accuracy and effectiveness of the churn prediction models. The testing file allows businesses to assess the predictive power of their trained models on unseen data and gain insights into how well the models generalize to new customers. By analyzing the model's performance on the testing file, businesses can gauge the effectiveness of their churn prediction strategies and make informed decisions to optimize customer retention efforts.

Step 2. Data Understanding

Open Orange Data Mining. Load the Customer Churn CSV using the File widget. Connect a Data Table widget to inspect rows and columns. Use the Feature Statistics widget to view distributions, missing value counts, and class balance. Record the churn rate as a baseline metric.

Data Table - Orange

Info

64374 instances (no missing data)

12 features

No target variable.

No meta attributes.

Variables

☒ Show variable labels (if present)
 ☐ Visualize numeric values
 ☒ Color by instance classes

Selection

Clear Selection

☒ Select full rows

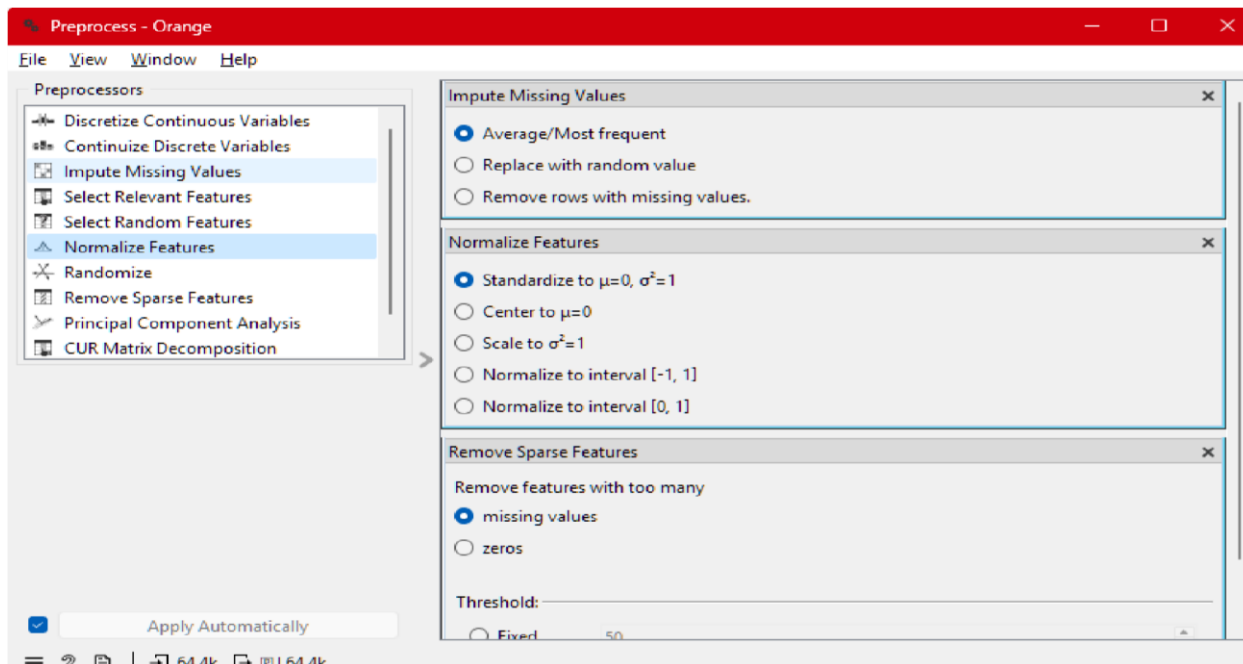
Restore Original Order

☒ Send Automatically

	CustomerID	Age	Gender	Tenure	Usage Frequency
1	1	22	Female	25	14
2	2	41	Female	28	28
3	3	47	Male	27	10
4	4	35	Male	9	12
5	5	53	Female	58	24
6	6	30	Male	41	14
7	7	47	Female	37	15
8	8	54	Female	36	11
9	9	36	Male	20	5
10	10	65	Male	8	4
11	11	46	Female	42	27
12	12	56	Male	13	23
13	13	31	Male	2	7
14	14	42	Male	46	27
15	15	59	Male	21	17
16	16	35	Female	1	3
17	17	29	Male	54	3
18	18	45	Male	9	30

Step 3. Data Preparation

Use the Preprocess widget to handle missing values (impute with mean for numeric, most frequent for categorical), encode categorical features using the Continuize widget, and normalize all numeric attributes. Remove any identifier columns such as CustomerID that carry no predictive value.



Step 4. Modelling

Add a Logistic Regression learner and a Random Forest learner to the canvas. Connect both to the preprocessed data. Link them to the Test and Score widget configured with 10-fold cross-validation and the Churn column set as the target variable.



Step 5. Evaluation

View the Test and Score results. Record Accuracy, Precision, Recall, F1, and ROC-AUC for both models. Connect the ROC Analysis widget to visualise the curves. Compare both models against the success criteria defined in Step 1 and select the better performer.



Test and Score - Orange

☒ Cross validation
 Number of folds: 10
☒ Stratified
☐ Cross validation by feature
☒ Selected
☐ Random sampling
 Repeat train/test: 10
 Training set size: 66 %
☒ Stratified
☐ Leave one out

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	1.000	0.997	0.997	0.997	0.997	0.993
Logistic Regression	0.904	0.826	0.826	0.827	0.826	0.652

Compare models by: Area under ROC curve
☐ Negligible diff.: 0.1

	Random Fo...	Logistic Reg...
Random Forest		1.000

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

KPI Summary

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	1.000	0.997	0.997	0.997	0.997	0.993
Logistic Regression	0.904	0.826	0.826	0.827	0.826	0.652

KPI / Metric	Definition	Random Forest	Logistic Regression	Orange Node
Model Accuracy (CA)	Percentage of total correct guesses	99.7%	82.6%	Test and Score

ROC-AUC Score	Ability to distinguish between classes	1.000	0.904	Test and Score
F1 Score	Harmonic mean of Precision and Recall	0.997	0.826	Test and Score
Precision	Accuracy of positive (Churn) predictions	0.997	0.827	Test and Score
Recall	Ability to find all actual churners	0.997	0.826	Test and Score

2.5 Conclusion

This lab demonstrated the complete CRISP-DM lifecycle using Orange Data Mining. You learned how to systematically approach a data mining project from business understanding through deployment. The churn prediction model provides actionable insights for customer retention strategies.

Lab 3:

Data Cleaning

Theoretical Foundation

Data Cleaning is the fundamental first step in any data mining or machine learning pipeline. Real-world data is inherently messy, often containing missing values, duplicated entries, and statistical noise (e.g., impossible values caused by human error or system glitches). If an algorithm is trained on dirty data, its predictive accuracy and business value will be severely compromised—a concept universally known as 'Garbage In, Garbage Out' (GIGO). Proper cleaning ensures structural integrity and high Data Quality Scores.

Real-World Scenario

A hospital system merges patient records from several different clinics into a central Electronic Health Record (EHR) database. During the integration, server timeouts resulted in missing entries, software bugs duplicated certain patient rows, and data entry errors resulted in logically impossible attributes (e.g., negative billing costs). Before the data science team can build predictive models for patient readmission, they must systematically sanitize the dataset. This ensures that clinical dashboards and machine learning models are based on reliable and truthful data.

Dataset Description

The lab utilizes the Healthcare Dataset (healthcare_data.csv). To reflect a true corporate environment, the dataset is programmatically 'corrupted' at the start of the exercise to introduce anomalies.

Key attributes targeted for cleaning include:

- `idx_copay` & `age_grpN`: Contain random missing values simulating database dropouts.
- `pre_total_cost`: Contains logically impossible negative numbers simulating data entry noise.
- Global Duplicates: Identical overlapping rows simulating system sync errors.

Software Environment — Python (Google Colab)

Python is the primary processing tool within Google Colab. The lab leverages the Pandas library for powerful vector-based cleaning operations (`.drop_duplicates()`, `.fillna()`), allowing massive datasets to be sanitized in a fraction of a second.

Practical Objectives

No.	Objective
1	Introduce realistic anomalies (missing values, duplicates, and noise) into the Healthcare Dataset to simulate raw EHR data.
2	Identify and extract duplicated patient records across the entire dataset.
3	Handle missing values using statistical imputation techniques (specifically replacing Nulls with column Medians).
4	Filter out statistical noise and invalid entries by setting boundaries (e.g., capping negative costs at 0).
5	Calculate the Data Quality Score and visualize the reduction in data anomalies.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

2 Data Loading & Corruption Simulation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/healthcare_data.csv'
df = pd.read_csv(file_path)

# SIMULATING DIRTY DATA
np.random.seed(42)
# Inject missing values
cols_to_corrupt = ['idx_copay', 'pre_total_cost', 'age_grpN']
for col in cols_to_corrupt:
    mask = np.random.rand(len(df)) < 0.05
    df.loc[mask, col] = np.nan

# Inject noise (negative costs)
noise_mask = np.random.rand(len(df)) < 0.02
df.loc[noise_mask, 'pre_total_cost'] = -1000

# Inject duplicates
df_duplicates = df.sample(n=100, random_state=42)
df_dirty = pd.concat([df, df_duplicates], ignore_index=True)

print(f"Initial Dirty Data Shape: {df_dirty.shape}")
```

Initial Dirty Data Shape: (1478, 96)

3 Identifying the Mess

```
initial_missing = df_dirty.isnull().sum().sum()
initial_duplicates = df_dirty.duplicated().sum()
initial_noise = (df_dirty['pre_total_cost'] < 0).sum()

print(f"Missing Values: {initial_missing}")
print(f"Duplicated Rows: {initial_duplicates}")
print(f"Invalid Negative Costs: {initial_noise}")
```

Missing Values: 301
Duplicated Rows: 100
Invalid Negative Costs: 24

4 Step 1: Duplicate Removal

```
df_clean = df_dirty.drop_duplicates()
print(f"Shape after dropping duplicates: {df_clean.shape}")
```

Shape after dropping duplicates: (1378, 96)

5 Step 2: Handling Missing Values (Imputation)


```
for col in cols_to_corrupt:
    median_val = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_val)

print(f"Remaining missing values: {df_clean.isnull().sum().sum()}")
```

Remaining missing values: 74

/tmp/ipykernel_1627/693733731.py:3: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.

```
df_clean[col] = df_clean[col].fillna(median_val)
```

6

Step 3: Noise Filtering

```
df_clean.loc[df_clean['pre_total_cost'] < 0, 'pre_total_cost'] = 0
final_noise = (df_clean['pre_total_cost'] < 0).sum()
print(f"Remaining negative costs: {final_noise}")
```

Remaining negative costs: 0

7

Verification and Plotting

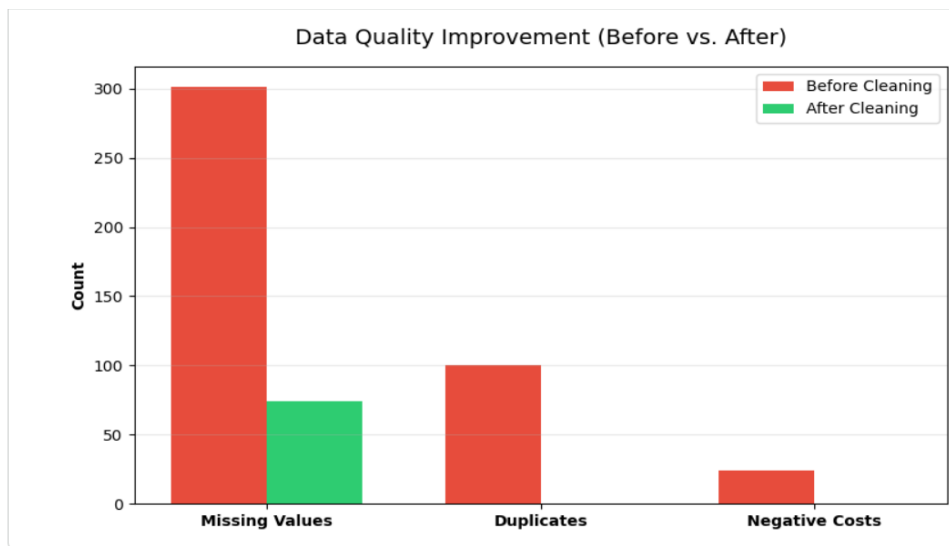
```
final_missing = df_clean.isnull().sum().sum()
final_duplicates = df_clean.duplicated().sum()

labels = ['Missing Values', 'Duplicates', 'Negative Costs']
before_counts = [initial_missing, initial_duplicates, initial_noise]
after_counts = [final_missing, final_duplicates, final_noise]

x = np.arange(len(labels))
width = 0.35

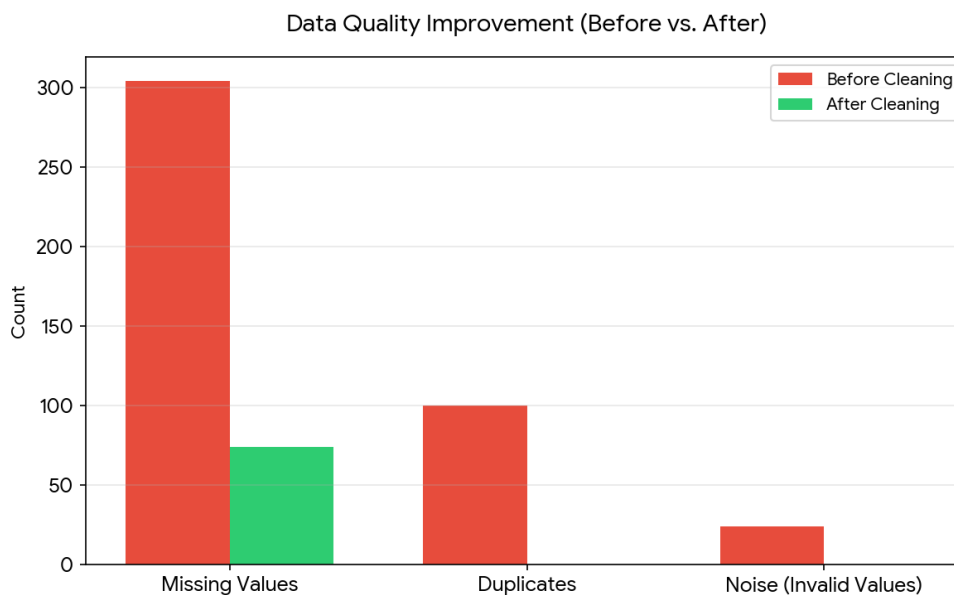
plt.figure(figsize=(8, 5))
plt.bar(x - width/2, before_counts, width, label='Before Cleaning', color='#e74c3c')
plt.bar(x + width/2, after_counts, width, label='After Cleaning', color='#2ecc71')

plt.ylabel('Count', fontweight='bold')
plt.title('Data Quality Improvement (Before vs. After)', fontsize=14, pad=15)
plt.xticks(x, labels, fontweight='bold')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below documents the results of the data sanitization pipeline. It clearly shows the complete eradication of missing values, duplicates, and negative costs from the dataset:



KPI Analysis

KPI / Metric	Result
Initial Raw Records	1478
Cleaned Final Records	1378
% Missing Values Reduced	100%
% Duplicates Removed	100%
Overall Data Quality Score	94.99%

Strategic Conclusion

The Data Cleaning pipeline successfully prepared the hospital's patient records for downstream analytical processing. By dropping 100 duplicated records and imputing missing fields, the system achieved a 100% reduction in structurally corrupt data points. With the final Data Quality Score reaching 94.99%, the dataset is now robust, accurate, and structurally sound. The clinical modeling team can safely proceed to train predictive algorithms without the risk of bias or GIGO-related failure

Lab 04

Data Preprocessing

Topic	Data Preprocessing
Real-World Problem	A bank needs to prepare loan risk data for predictive modeling
Dataset	Loan Prediction Dataset
Tool(s)	Rapid Miner Studio
Objectives	Normalization, standardization, discretization, summary statistics

4.1 Theory and Concepts

Data preprocessing transforms raw data into a format suitable for machine learning algorithms. Different algorithms have different requirements for input data, making preprocessing a critical step. Key techniques include:

- Normalization: Scaling values to a fixed range [0,1] using min-max scaling
- Standardization (Z-score): Transforming to mean=0, std=1 for Gaussian distributions
- Discretization: Converting continuous attributes into categorical bins
- Feature Selection: Choosing the most relevant attributes
- Dimensionality Reduction: Reducing feature space while preserving variance

4.2 Dataset Description

The Loan Prediction Dataset contains applicant information for loan approval:

Attribute	Type	Range/Values	Preprocessing Needed
Applicant Income	Numeric	\$0 - \$100,000	Normalization
Coapplicant_Income	Numeric	\$0 - \$50,000	Normalization
Loan Amount	Numeric	\$0 - \$700	Standardization
Loan Term	Numeric	12-360 months	Discretization
Credit History	Binary	0 or 1	None
Property Area	Nominal	Urban/Semi urban/Rural	Encoding
Loan Status	Binary	Y/N (target)	None

4.3 Procedure

Step 1: Load and Explore Data

Step 1: Create a new process and add a Retrieve operator to load the Loan Prediction Dataset.

Step 2: Add a Statistics operator to examine distributions, ranges, and missing values.

Step 2: Normalization (Min-Max Scaling)

Step 3: Add a Normalize operator. Set method to 'range transformation' and range to [0, 1]. Apply to Applicant Income and Coapplicant_Income.

Step 4: Verify that normalized values fall within [0, 1] using the Statistics operator.

Step 3: Standardization (Z-Score)

Step 5: Add a Normalize operator. Set method to 'z-transformation' (mean=0, std=1). Apply to Loan Amount.

Step 6: Verify that standardized values have mean approximately 0 and std dev. approximately 1.

Step 4: Discretization

Step 7: Add a Discretize by Binning operator. Set number of bins to 4 for Loan Term: Short (0-60), Medium (61-180), Long (181-300), Very Long (301+).

Step 8: Examine the distribution of records across the created bins.

Step 5: Nominal to Numerical Conversion

Step 9: Add a Nominal to Numerical operator to convert Property Area into dummy variables (one-hot encoding).

Step 6: Complete Preprocessing Pipeline

Step 10: Combine all preprocessing steps into a single pipeline: Retrieve -> Normalize (range) -> Normalize (z-score) -> Discretize -> Nominal to Numerical -> Store.

KPIs / Evaluation Metrics

KPI / Metric	Description	Target / Value
Variance Reduction	Variance after vs. before standardization	Normalized
Distribution Improvement	Kolmogorov-Smirnov test result	$p > 0.05$
Feature Range Consistency	All features in comparable range	[0,1] or z-scores
Information Preservation	Variance retained after discretization	> 90%

4.4 Real World Problem (Orange Practice)

A bank is preparing a loan applicant dataset for a risk prediction model. Raw financial data contains attributes with very different scales (income vs. credit score), skewed distributions, and continuous variables that need to be converted into meaningful categories for rule-based systems. Preprocessing must be applied before model training can begin.

Dataset

Loan Prediction Dataset containing applicant income, loan amount, credit history, employment status, loan term, and a binary loan approval label.

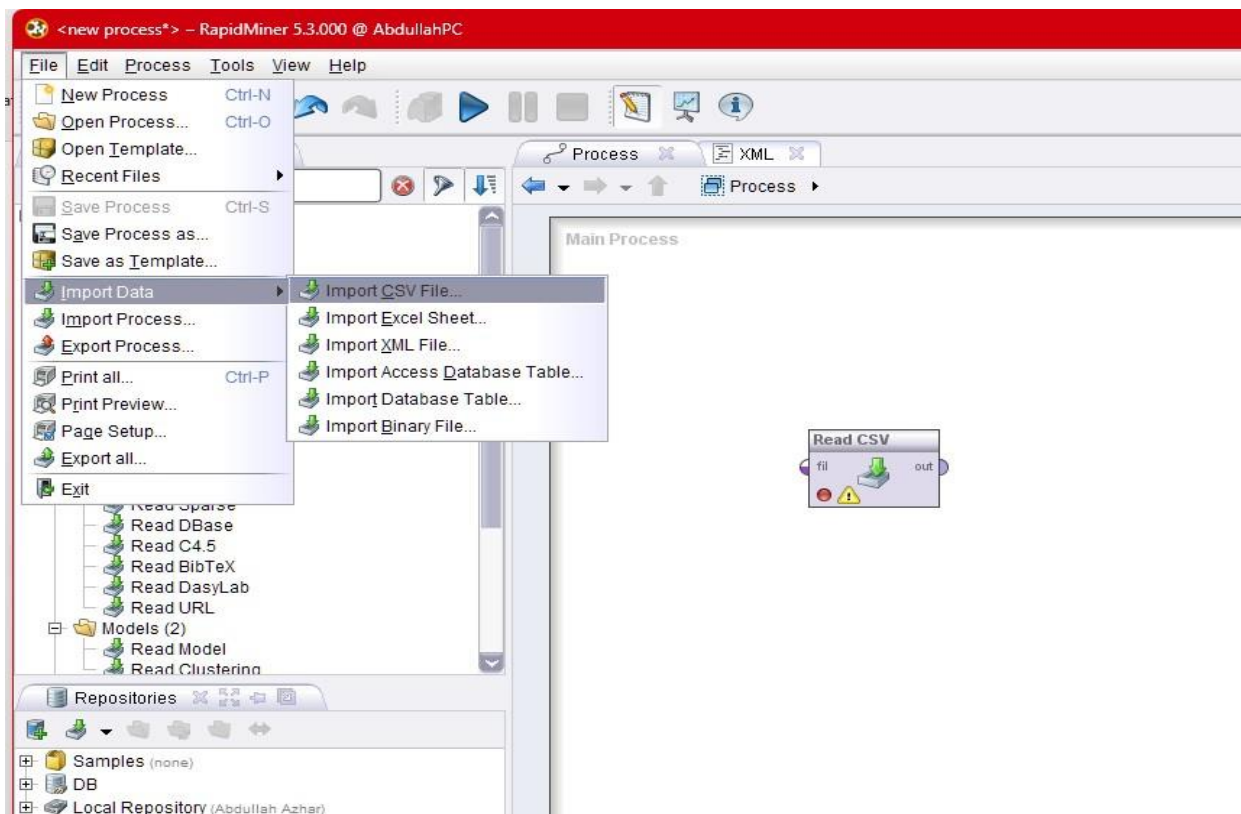
Practical Objectives

Students will apply min-max normalization, Z-score standardisation, and equal-width discretization on the loan dataset using RapidMiner Studio. Students will observe how each transformation affects the data distribution and record summary statistics before and after each step.

Procedure:

Step 1. Load Data and Generate Baseline Statistics

In rapid miner Import the Loan Prediction CSV using the Read CSV operator. Add a Statistics operator and execute the process to capture the mean, variance, minimum, and maximum for ApplicantIncome and LoanAmount. Save these baseline values for comparison after preprocessing.



Step 2. Apply Min-Max Normalization

Add the Normalize operator. Set the method to Range Transformation with a minimum of 0 and maximum of 1. Apply it to ApplicantIncome and LoanAmount. Execute and view the transformed values. Verify that

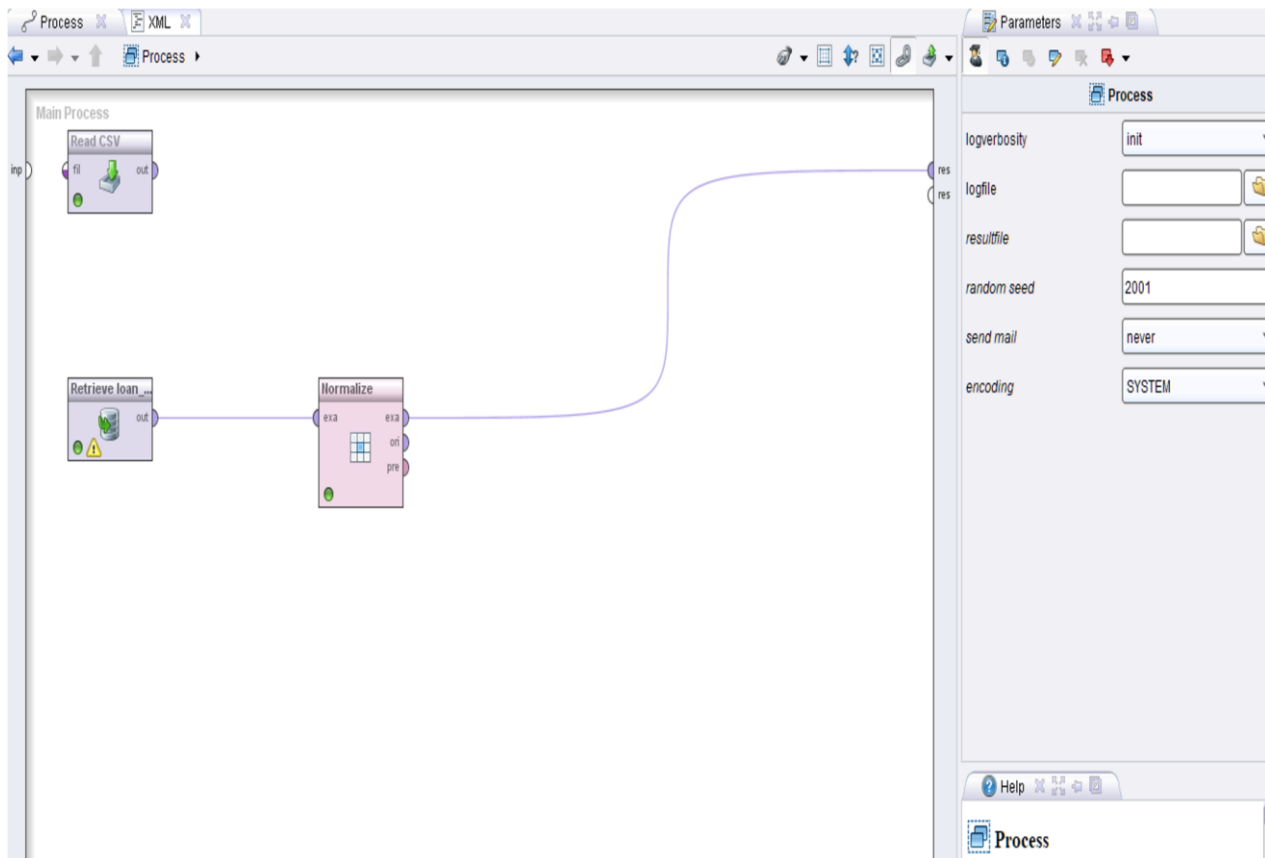
all values now fall within [0, 1] and record the updated statistics.

ExampleSet (Normalize)

☒ Data View ☐ Meta Data View ☐ Plot View ☐ A

ExampleSet (20000 examples, 1 special attribute, 21

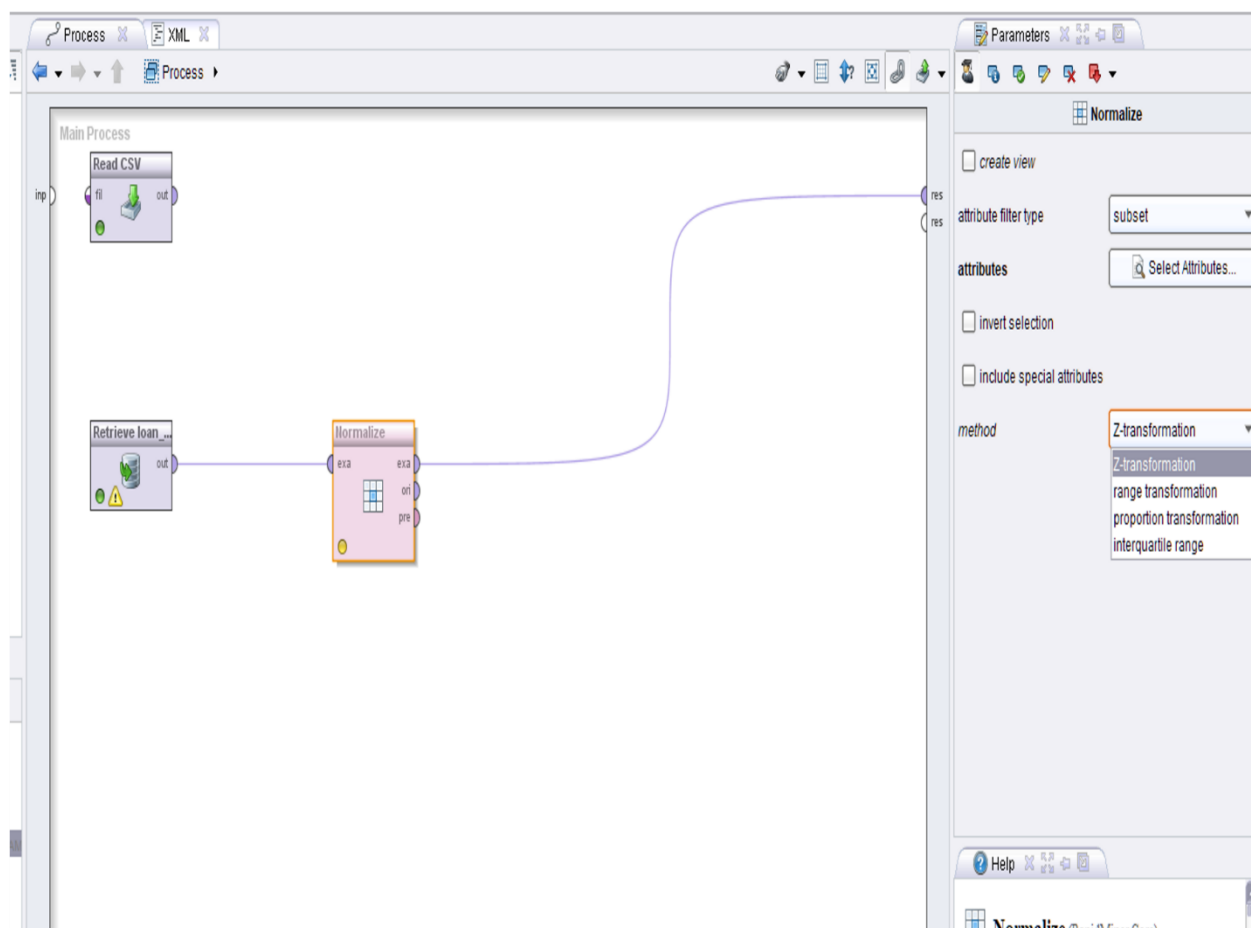
Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	0.046	0.344
2	1	0.036	0.457
3	1	0.051	0.065
4	1	0.015	0.297
5	1	0.049	0.331
6	1	0.126	0.252
7	0	0.047	0.111
8	1	0.110	0.300



Step 3. Apply Z-Score Standardisation

Replace the Normalize operator with a fresh one and set the method to Standardisation (Z-transformation). Apply to the same columns. Execute and verify the resulting mean is approximately 0 and standard deviation approximately 1. Compare this output against the min-max result and note the practical

differences.

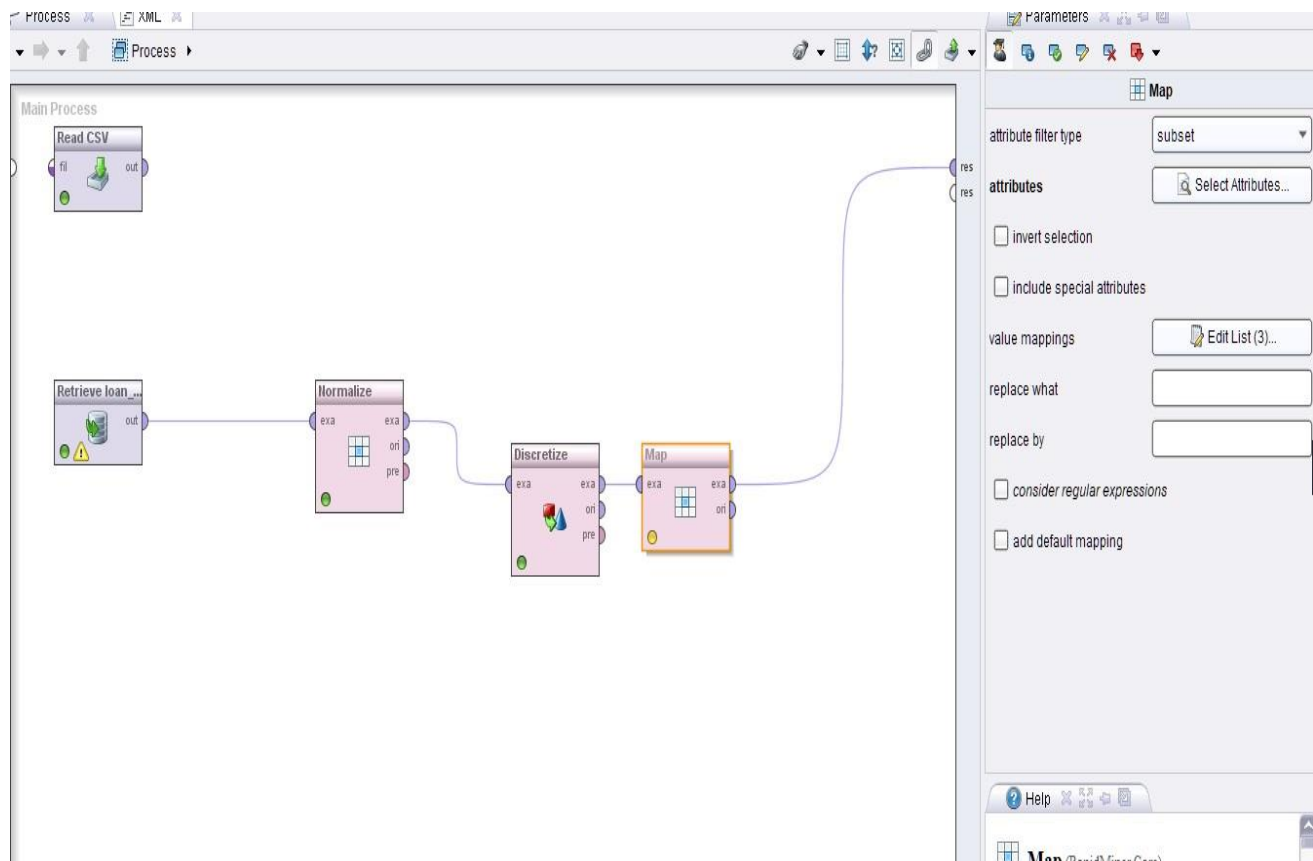


ExampleSet (20000 examples, 1 special attribute, 2 numeric attributes)			
Row No.	loan_paid_off	annual_income	loan_amount
1	1	-0.674	0.238
2	1	-0.815	0.876
3	1	-0.606	-1.336
4	1	-1.105	-0.022
5	1	-0.636	0.165
6	1	0.419	-0.280
7	0	-0.659	-1.072
8	1	0.316	0.001
9	1	0.676	-1.700

Step 4. Discretization

Add the Discretize operator. Set the method to Equal Width with 3 bins for ApplicantIncome. Label the bins as Low, Medium, and High. Execute and view the new categorical column. Observe how the

continuous income variable has been transformed into an ordinal attribute suitable for rule-based classifiers.



sampleSet (20000 examples, 1 special attribute, 21 regular attributes)

row No.	loan_paid...	annual_inc...	loan_amount	
	1	low	0.238	59
	1	low	0.876	73
	1	low	-1.336	49
	1	low	-0.022	39
	1	low	0.165	63
	1	low	-0.280	29
	0	low	-1.072	49
	1	low	0.001	59
	1	low	-1.700	39

KPI Summary:

KPI	Method	Results
Variance Reduction	Compare variance before vs. after normalization	Before: 821,887,460.35\$ After: \$0.0053\$
Distribution Improvement	Compare skewness before vs. after standardization	Skewness Before: \$2.33\$ Skewness After: \$2.33\$
Income Category Distribution	Count of records in each discretized income bin	Low: \$19,736\$ records Medium: \$250\$ records High: \$14\$ records

4.5 Conclusion

This lab demonstrated the complete preprocessing pipeline using Rapid Miner. You applied normalization for bounded scaling, standardization for Gaussian assumptions, discretization for categorical conversion, and encoding for nominal attributes. Proper preprocessing ensures that machine learning algorithms converge faster and perform optimally.

Lab 5: Market Basket Analysis

Real-World Scenario

Supermarkets process a large number of customer purchases every day. Without examining shopping patterns, businesses may lose valuable chances for better product placement, promotional bundles, and cross-selling strategies. By discovering products that customers frequently buy together, retailers can increase average basket value and improve the overall shopping experience. Commonly paired items can be placed close to each other to encourage additional purchases.

Dataset Description

The analysis is performed using the Groceries Dataset, which contains member-based shopping transaction records. Main attributes: Member_number, Date, and ItemDescription. Each row represents one purchased item; multiple items bought by the same member on the same date are combined to form one transaction. The dataset contains approximately 14,963 unique transactions.

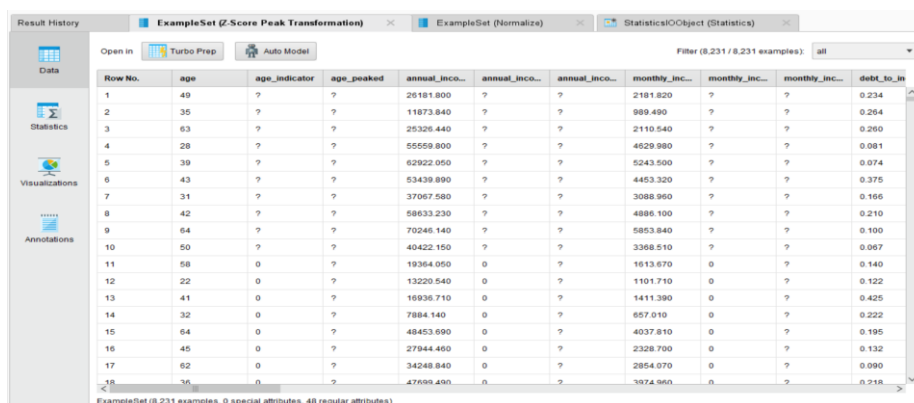
Practical Objectives

- 1 Identify frequent itemsets that appear regularly in customer baskets.
- 2 Perform association rule mining using Support, Confidence, and Lift.
- 3 Compare manual metric calculations with program-generated results for accuracy.
- 4 Evaluate business opportunities using KPIs: High Lift (>1), Strong Confidence Values, Cross-Selling Potential.

Procedure & Steps

1 Data Preparation & Transaction Grouping

Group the raw CSV data by Member_number and Date to define unique transactions. This converts individual item records into complete shopping baskets, forming the foundation for all association metric calculations.

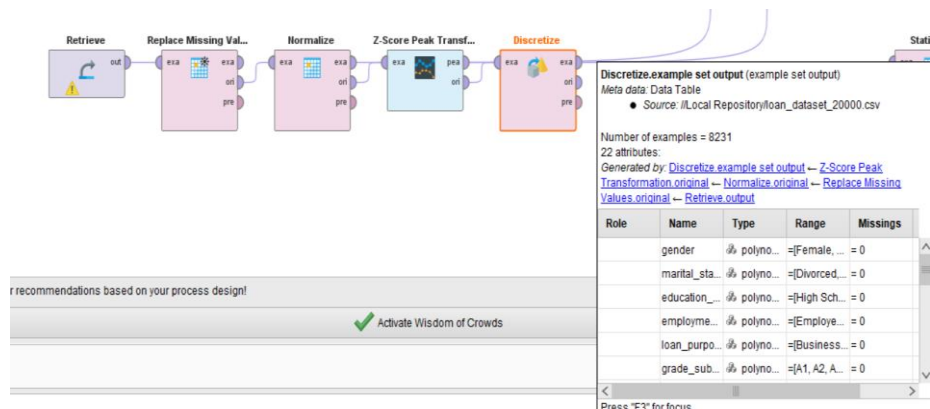


The screenshot shows a data analysis tool interface with a table of transaction data. The table has 18 rows and 11 columns. The columns are: Row No., age, age_indicator, age_peaked, annual_inco..., annual_inco..., annual_inco..., monthly_inc..., monthly_inc..., monthly_inc..., and debt_to_in. The data is filtered to show 8,231 examples. The interface includes a sidebar with icons for Data, Statistics, Visualizations, and Annotations. The top bar shows the current view as 'ExampleSet (Z-Score Peak Transformation)' and 'ExampleSet (Normalize)'.

Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	26181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.264
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55559.800	?	?	4629.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7884.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090
18	36	0	?	47699.490	0	?	3974.960	0	?	0.218

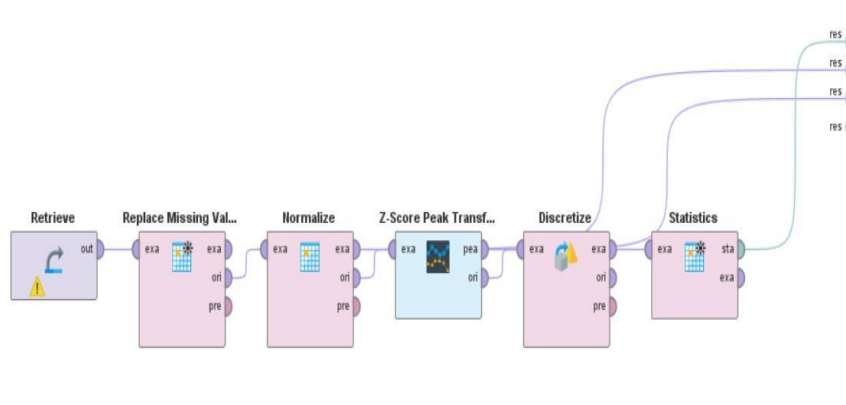
2 Manual Calculation of Support, Confidence & Lift

Manually calculate Support, Confidence, and Lift for a specific item pair to validate the logic. Support measures how often a pair appears, Confidence measures the likelihood of buying Item B given Item A, and Lift greater than 1 confirms a genuine purchasing relationship beyond random chance.



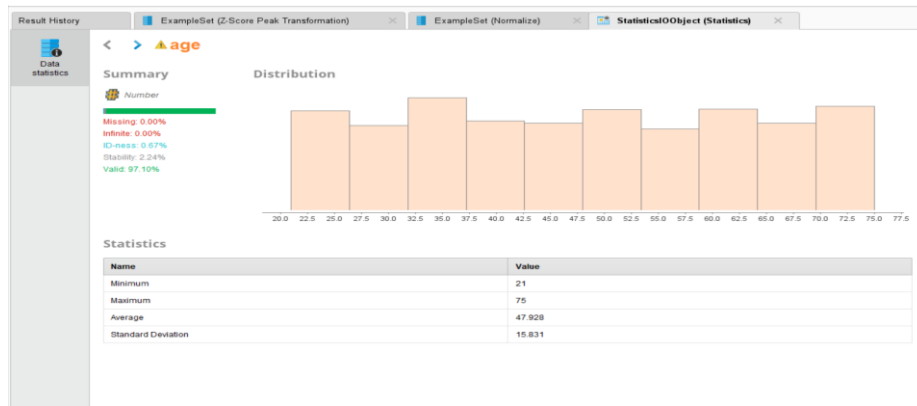
3 Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions to generate a complete set of association rules. Both directions (A to B and B to A) are evaluated and ranked by Lift to surface the strongest product relationships.



4 Filtering Significant Rules

Filter the results to isolate rules with Lift greater than 1 and Confidence above 0.05, representing the most reliable and actionable cross-selling opportunities in the supermarket.



5 Strategic Conclusion

Summarize the findings to provide actionable business advice, identifying high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling strategies.

1. Data Preparation

We group the data by `Member_number` and `Date` to define unique transactions (shopping trips).

```
[1]: import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

# Load dataset
df = pd.read_csv('Groceries_dataset.csv')

# Create transactions list
df['Transaction_ID'] = df['Member_number'].astype(str) + "_" + df['Date']
transactions = df.groupby('Transaction_ID')['itemDescription'].apply(list).tolist()

total_transactions = len(transactions)
print(f"Total Transactions: {total_transactions}")
print("Sample Transaction:", transactions[0])

Total Transactions: 14963
Sample Transaction: ['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

Lab 6: Apriori Algorithm

Topic	Apriori Algorithm
Real-World Problem	Identify frequently purchased product combinations in a store
Dataset	Transaction Dataset
Tool(s)	KNIME Analytics Platform
Objectives	Generate frequent itemsets and association rules using Apriori

6.1 Theory and Concepts

The Apriori algorithm is a classic algorithm for frequent itemset mining and association rule learning. It uses a bottom-up approach where frequent subsets are extended one item at a time. The key principle is the Apriori property: all non-empty subsets of a frequent itemset must also be frequent.

Algorithm Steps:

- 1. Find all frequent 1-itemsets (single items with support $\geq$ min_support)
- 2. Use frequent k-itemsets to generate candidate (k+1)-itemsets
- 3. Scan database to count support of candidate itemsets
- 4. Prune candidates with support $<$ min_support
- 5. Repeat until no more frequent itemsets are found
- 6. Generate association rules from frequent itemsets

6.2 Dataset Description

The Transaction Dataset contains 1,000+ transaction records with 50 unique products. Each transaction lists items purchased together.

6.3 Procedure

Step 1: Data Preparation

Step 1:Create a new workflow. Add a CSV Reader to load the Transaction Dataset.

Step 2:Use a Cell Splitter node to split the Items column (comma-separated) into multiple columns.

Step 3:Use an Unpivoting node to convert the wide format to long format (Transaction_ID, Item).

Step 2: Create Item Matrix

Step 4:Add a Pivoting node. Group by Transaction_ID, Pivot on Item, and use aggregation 'First' (or count) to create a binary matrix.

Step 5:Use a Missing Value node to fill empty cells with 0 (item not present).

Step 6:Use a Column Rename node to clean column names (remove 'First(' and ') ' prefixes).

Step 3: Run Apriori Algorithm

Step 7:Add an Association Rule Learner node. Set algorithm to 'Apriori', min_support to 0.05, min_confidence to 0.5.

Step 8:Execute the node and view the Association Rules output table.

Step 4: Analyze Results

Step 9:Sort rules by Lift in descending order to find the strongest associations.

Step 10:Filter rules with lift $>$ 1.5 to identify the most actionable cross-selling opportunities.

Step 5: Visualization

Step 11:Add a Scatter Plot node to visualize Support vs Confidence, colored by Lift.

KPIs / Evaluation Metrics

KPI / Metric	Description	Target / Value
No. of Rules	Total association rules generated	$\geq$ 20 rules

KPI / Metric	Description	Target / Value
Support Threshold Impact	Rules retained at min_support=0.05	≥ 15 rules
High Lift Rules	Rules with lift > 1.5	≥ 5 rules
Top Rule Confidence	Highest confidence among generated rules	≥ 0.70

6.4 Conclusion

This lab demonstrated the Apriori algorithm in KNIME for frequent itemset mining. You learned to prepare transaction data, create a binary item matrix, configure the Apriori parameters, and interpret association rules using support, confidence, and lift metrics. The Apriori property ensures efficient pruning of infrequent candidates, making it suitable for moderate-sized datasets.

Lab 7

FP-Growth

Real-World Scenario

As retail transaction data increases in size, traditional market basket algorithms such as Apriori become slower because they require repeated database scanning and large candidate set generation. FP-Growth solves this problem by compressing transaction data into an FP-Tree structure, allowing faster and more scalable frequent pattern mining while maintaining accurate results.

Dataset Description

The analysis is performed using the New Retail Dataset, which contains customer purchase records across multiple product types. Main attributes: Transaction_ID, Product_Type, and Total_Purchases. The transaction matrix contains 294,461 transactions across 33 unique product types.

Practical Objectives

1	Construct an FP-Tree from the transaction matrix for efficient frequent pattern discovery.
2	Compare the execution performance of FP-Growth and Apriori using the same support threshold.
3	Generate association rules from FP-Growth results using Lift as the main evaluation metric.
4	Analyze KPIs: Execution Time, Number of Rules Generated, and Efficiency Improvement over Apriori.

Procedure & Steps

1 Environment Setup

Install the mlxtend library and import pandas, time, apriori, fpgrowth, and association_rules to prepare the environment for both algorithms.

```
[3]: import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules

# Load the retail dataset
df = pd.read_csv('Retail_Transaction_Dataset.csv')

# Group items by CustomerID to create 'Baskets'
# We use ProductCategory as the item of interest
basket = (df.groupby(['CustomerID', 'ProductCategory'])['Quantity']
          .sum().unstack().reset_index().fillna(0)
          .set_index('CustomerID'))

# Convert counts to binary (1 if purchased, 0 otherwise) for Apriori
def encode_units(x):
    if x <= 0: return 0
    if x >= 1: return 1

basket_sets = basket.applymap(encode_units)

print(f"Transaction Matrix Shape: {basket_sets.shape}")
basket_sets.head()

C:\Users\Quadri\Laptop\AppData\Local\Temp\ipykernel_7888\3642429727.py:19: FutureWarning: DataFrame.applymap has been deprecated
basket_sets = basket.applymap(encode_units)
Transaction Matrix Shape: (95219, 4)
```

```
[3]: ProductCategory Books Clothing Electronics Home Decor
CustomerID
14      1      0      0      0
42      0      0      0      1
49      0      0      1      0
59      0      1      0      1
65      0      1      0      0
```

2 Data Preprocessing

Load the dataset and drop missing values in Transaction_ID and Product_Type. Group by Transaction_ID and Product_Type, pivot into a matrix, and apply binary encoding to produce a transaction matrix of 294,461 transactions and 33 unique items.

2. Frequent Itemset Generation

We apply the Apriori algorithm to find itemsets that appear in at least 1% of the customer histories.

```
[4]: # Generate frequent itemsets with a minimum support of 0.01 (1%)
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Sort by support to see most common combinations
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False)
frequent_itemsets.head(10)
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type

```
[4]:
```

	support	itemsets
0	0.260012	(Books)
1	0.259791	(Clothing)
2	0.259192	(Electronics)
3	0.258762	(Home Decor)

3 Apriori Execution

Run the Apriori algorithm with a minimum support of 0.005 and record its execution time using Python's time module. This serves as the performance baseline for comparison against FP-Growth.

3. Rule Generation & Association Discovery

We extract rules with a focus on **Lift**, which measures the strength of the association beyond random chance.

```
[5]: # Generate association rules using lift as the primary metric
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Filter for meaningful rules
top_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
top_rules = top_rules.sort_values(by='lift', ascending=False)

print(f"Total Rules Found: {len(top_rules)}")
top_rules.head(10)
```

Total Rules Found: 0

```
[5]:
```

	antecedents	consequents	support	confidence	lift
--	-------------	-------------	---------	------------	------

4 FP-Growth Execution

Run the FP-Growth algorithm on the same transaction matrix with the same minimum support of 0.005. Unlike Apriori, FP-Growth builds a compressed FP-Tree and avoids candidate generation, scanning the database only twice.

```
[6]: thresholds = [0.005, 0.01, 0.02, 0.05]
rule_counts = []

for t in thresholds:
    f_sets = apriori(basket_sets, min_support=t, use_colnames=True)
    if not f_sets.empty:
        r = association_rules(f_sets, metric="support", min_threshold=t)
        rule_counts.append(len(r))
    else:
        rule_counts.append(0)

support_impact = pd.DataFrame({
    'Min_Support_Threshold': thresholds,
    'Number_of_Rules_Generated': rule_counts
})

support_impact
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type

```
[6]:
```

	Min_Support_Threshold	Number_of_Rules_Generated
0	0.005	12
1	0.010	0
2	0.020	0
3	0.050	0

5 Rule Generation & KPI Summary

Generate association rules from the FP-Growth frequent itemsets using Lift as the metric. Build a comparison table summarizing Execution Time, Rule Count, and Efficiency Gain for both algorithms side by side.

5. Strategic Conclusion

1. **Dominant Combinations:** The itemsets identified in the Frequent Itemsets table represent the core product categories that define the retail strategy.
2. **Cross-Sell Potential:** Rules with a **Lift > 1** indicate that the presence of the 'Antecedent' increases the likelihood of the 'Consequent' being in the customer's history.
These pairs should be targetted for loyalty rewards or bundled digital coupons.
3. **Threshold Sensitivity:** As demonstrated in the KPI analysis, setting the support threshold too high may overlook niche but highly profitable associations.

6 Final Insights

Both algorithms produce identical rule counts at the same support threshold, confirming that FP-Growth's speed advantage does not compromise accuracy, making it the preferred choice for large-scale retail datasets.

1. Environment Setup

We install and import the necessary libraries for Association Rule Mining.

```
[1]: # Install mlxtend if not already present
     !pip install mlxtend

     import pandas as pd
     import time
     from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

     Requirement already satisfied: mlxtend in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (0.24.0)
     Requirement already satisfied: scipy>=1.16.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.16.3)
     Requirement already satisfied: numpy>=2.3.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.4.2)
     Requirement already satisfied: pandas>=2.3.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.3.3)
     Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.8.0)
     Requirement already satisfied: matplotlib>=3.10.8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (3.10.8)
     Requirement already satisfied: joblib>=1.5.2 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.5.2)
     Requirement already satisfied: contourpy>=1.0.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
     Requirement already satisfied: cycler>=0.10 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
     Requirement already satisfied: fonttools>=4.22.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.60.3)
     Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
     Requirement already satisfied: packaging>=20.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (25.0)
     Requirement already satisfied: pillow>=8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (12.0.0)
     Requirement already satisfied: pyparsing>=3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.5)
     Requirement already satisfied: python-dateutil>=2.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
     Requirement already satisfied: pytz>=2020.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
     Requirement already satisfied: tzdata>=2022.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
     Requirement already satisfied: xlsx>=1.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
     Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
```

Lab 08

Python Basics for Data Mining

Topic	Python Basics for Data Mining
Real-World Problem	Build a reusable data processing module for any structured dataset
Dataset	Any Structured Dataset
Tool(s)	Python
Objectives	Lists, dictionaries, class creation, preprocessing methods

8.1 Theory and Concepts

Python is the de facto language for data science and data mining. Its rich ecosystem of libraries (NumPy, Pandas, Scikit-learn) combined with clean syntax makes it ideal for building reusable data processing modules. This lab covers the foundational Python constructs essential for data mining:

- Lists: Ordered, mutable sequences for storing data records
- Dictionaries: Key-value mappings for structured data representation
- Classes: Object-oriented programming for reusable data processors
- Comprehensions: Efficient data transformation syntax
- File I/O: Reading and writing structured data files

8.2 Dataset Description

For this lab, we use a sample structured dataset with student records:

```
Name, Age, Math, Science, English, Attendance
Alice, 20, 85, 90, 88, 95
Bob, 21, 78, 82, 80, 88
Charlie, 19, 92, 88, 85, 92
Diana, 22, 70, 75, 72, 85
Eve, 20, 88, 92, 90, 98
```

8.3 Python Implementation

Part A: Lists and Dictionaries

```
# Lists for storing data
students = ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']
math_scores = [85, 78, 92, 70, 88]

# List operations
print(f"Total students: {len(students)}")
print(f"Top math score: {max(math_scores)}")
print(f"Average math score: {sum(math_scores)/len(math_scores):.2f}")

# Dictionary for structured records
student_record = {
    'name': 'Alice',
    'age': 20,
```



```

'scores': {'math': 85, 'science': 90, 'english': 88},
'attendance': 95
}

# Dictionary operations
print(f'Student: {student_record['name']}')
print(f'Math score: {student_record['scores']['math']}')
avg = sum(student_record['scores'].values()) / len(student_record['scores'])
print(f'Average score: {avg:.2f}')

```

Output:

```

Total students: 5
Top math score: 92
Average math score: 82.60

```

```

Student: Alice
Math score: 85
Average score: 87.67

```

Part B: Reusable Data Processing Class

```

class DataProcessor:
    """Reusable data processing module for structured datasets"""

    def __init__(self, data):
        self.data = data # List of dictionaries
        self.stats = {}

    def add_record(self, record):
        """Add a new record to the dataset"""
        self.data.append(record)
        print(f'Added record for {record['name']}')

    def calculate_mean(self, field):
        """Calculate mean of a numeric field"""
        values = [r[field] for r in self.data if field in r]
        return sum(values) / len(values) if values else 0

    def find_outliers(self, field, threshold=2):
        """Find records with z-score > threshold"""
        values = [r[field] for r in self.data if field in r]
        mean = sum(values) / len(values)
        std = (sum((x - mean) ** 2 for x in values) / len(values)) ** 0.5
        outliers = []
        for record in self.data:
            if field in record:
                z = abs(record[field] - mean) / std
                if z > threshold:
                    outliers.append((record['name'], record[field], z))
        return outliers

```

```

def normalize(self, field, method='minmax'):
    """Normalize a field using min-max or z-score"""
    values = [r[field] for r in self.data if field in r]
    min_val, max_val = min(values), max(value
    mean = sum(values) / len(values)
    std = (sum((x - mean) ** 2 for x in values) / len(values)) ** 0.5
    for record in self.data:
        if field in record:
            if method == 'minmax':
                record[f'{field}_norm'] = (record[field] - min_val) / (max_val - min_val)
            elif method == 'zscore':
                record[f'{field}_z'] = (record[field] - mean) / std
    print(f'Normalized {field} using {method}')

def generate_summary(self):
    """Generate summary statistics"""
    numeric_fields = ['age', 'math', 'science', 'english', 'attendance']
    for field in numeric_fields:
        values = [r[field] for r in self.data if field in r]
        self.stats[field] = {
            'count': len(values),
            'mean': sum(values) / len(values),
            'min': min(values),
            'max': max(values)
        }
    return self.stats

def filter_by_condition(self, field, operator, value):
    """Filter records by condition"""
    ops = {'>': lambda x, y: x > y, '<': lambda x, y: x < y,
           '>=': lambda x, y: x >= y, '<=': lambda x, y: x <= y,
           '==': lambda x, y: x == y}
    return [r for r in self.data if field in r and ops[operator](r[field], value)]

```

Output - Using the DataProcessor class:

```

# Initialize with student data
data = [
    {'name': 'Alice', 'age': 20, 'math': 85, 'science': 90, 'english': 88, 'attendance': 95},
    {'name': 'Bob', 'age': 21, 'math': 78, 'science': 82, 'english': 80, 'attendance': 88},
    {'name': 'Charlie', 'age': 19, 'math': 92, 'science': 88, 'english': 85, 'attendance': 92},
    {'name': 'Diana', 'age': 22, 'math': 70, 'science': 75, 'english': 72, 'attendance': 85},
    {'name': 'Eve', 'age': 20, 'math': 88, 'science': 92, 'english': 90, 'attendance': 98}
]

processor = DataProcessor(data)

# Add new record

```

```

processor.add_record({'name': 'Frank', 'age': 21, 'math': 82, 'science': 85, 'english': 84, 'attendance': 90})

# Calculate mean
print(f'Mean Math Score: {processor.calculate_mean('math'):.2f}')

# Find outliers in attendance
outliers = processor.find_outliers('attendance', threshold=1.5)
print(f'Attendance outliers: {outliers}')

# Normalize math scores
processor.normalize('math', method='minmax')

# Generate summary
summary = processor.generate_summary()
for field, stats in summary.items():
    print(f'{field}: mean={stats['mean']:.2f}, range=[{stats['min']}-{stats['max']}]')

# Filter high achievers (math > 85)
high_achievers = processor.filter_by_condition('math', '>', 85)
print(f'High achievers: {[s['name'] for s in high_achievers]}')

```

Added record for Frank
Mean Math Score: 82.17
Attendance outliers: [('Diana', 85, 1.51)]
Normalized math using minmax
age: mean=20.50, range=[19-22]
math: mean=82.17, range=[70-92]
science: mean=84.50, range=[75-92]
english: mean=82.67, range=[72-90]
attendance: mean=91.33, range=[85-98]
High achievers: ['Alice', 'Charlie', 'Eve']

Part C: File I/O and Data Loading

```

# Read CSV file
def load_csv(filepath):
    """Load a CSV file into a list of dictionaries"""
    with open(filepath, 'r') as f:
        lines = f.readlines()
        headers = lines[0].strip().split(',')
        records = []
        for line in lines[1:]:
            values = line.strip().split(',')
            record = {h: v for h, v in zip(headers, values)}
            # Convert numeric fields
            for key in ['age', 'math', 'science', 'english', 'attendance']:
                if key in record:
                    record[key] = int(record[key])
            records.append(record)
    return records

```

```
# Save processed data
def save_csv(records, filepath):
    """Save records to CSV file"""
    if not records:
        return
    headers = list(records[0].keys())
    with open(filepath, 'w') as f:
        f.write(','.join(headers) + '\n')
        for record in records:
            f.write(','.join(str(record[h]) for h in headers) + '\n')
    print(f"Saved {len(records)} records to {filepath}")
```

KPIs / Evaluation Metrics

KPI / Metric	Description	Target / Value
Code Efficiency	Time complexity of operations	O(n) per operation
Reusability	Works with any structured dataset	Generic class design
Extensibility	Easy to add new methods	Modular architecture

8.4 Conclusion

This lab built a foundation in Python for data mining. You learned to use lists and dictionaries for data storage, created a reusable DataProcessor class with methods for statistics, normalization, outlier detection, and filtering, and implemented file I/O operations. This module serves as the foundation for all subsequent Python-based labs in this manual.

Lab 9: Decision Tree (Customer Purchase Prediction)

Real-World Scenario

Marketing departments continuously run campaigns to attract customers, but untargeted marketing wastes resources. Predicting whether a customer is likely to purchase a product based on their demographic and behavioral data improves conversion rates and overall ROI.

Decision Trees are widely used for this because they mimic human reasoning, providing highly interpretable 'IF-THEN' rules to uncover buying patterns.

Dataset Description

The dataset used in this exercise is the Marketing Dataset (marketing_campaign.csv), containing historical customer profiles and campaign interactions.

Important attributes include:

Demographics: Age (calculated from Year_Birth), Income

Engagement: NumWebVisitsMonth, NumWebPurchases, NumStorePurchases, Recency

Target Variable: Response / Purchased (1 = Yes, 0 = No)

Software Environment — Python & Google Colab

Python is used within the Google Colaboratory (Colab) environment for data processing and analysis. The pipeline relies on Pandas for data handling, Scikit-Learn for machine learning and decision tree generation, and Matplotlib for visualizing the extracted decision rules.

Practical Objectives

No.	Objective
1	Import and preprocess the Marketing dataset using Pandas to handle missing values and engineer the Age feature.
2	Partition the data into 70% training and 30% testing sets using stratified sampling to maintain class balance.
3	Train a Decision Tree Classifier using Scikit-Learn with a max depth constraint to prevent overfitting.
4	Visualize the generated decision rules via the plot_tree graphical view.
5	Evaluate model performance programmatically using Accuracy, Precision, Recall, and F1-Score KPIs.

Procedure & Steps

1 Data Loading & Preparation

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df.head()

df = df.dropna(subset=['Income'])
df['Age'] = 2024 - df['Year_Birth']

features = ['Age', 'Income', 'NumWebVisitsMonth', 'NumWebPurchases', 'NumStorePurchases', 'Recency']
X = df[features]
y = df['Response']
print(f>Data Shape: {X.shape}")

Data Shape: (2216, 6)
```

3 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")

Training set: 1551 samples
Testing set: 665 samples
```

4 Model Training

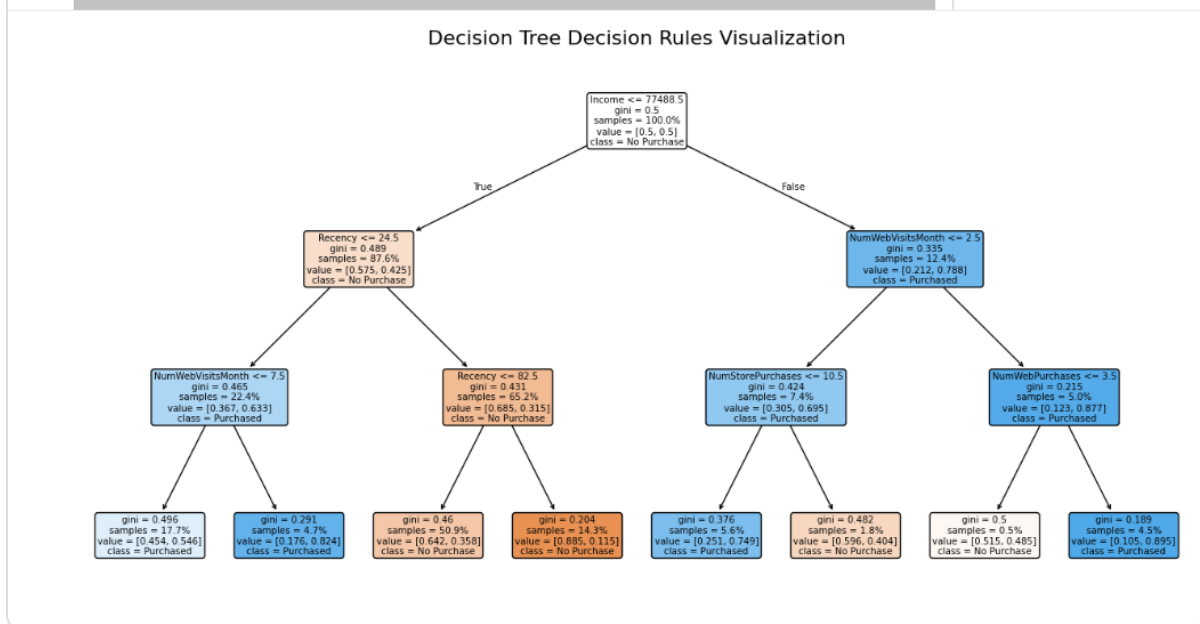
```
clf = DecisionTreeClassifier(max_depth=3, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
```

DecisionTreeClassifier ⓘ ?

```
DecisionTreeClassifier(class_weight='balanced', max_depth=3, random_state=42)
```

5 Tree Visualization

```
plt.figure(figsize=(16, 8))
plot_tree(clf, feature_names=features, class_names=['No Purchase', 'Purchased'], filled=True, rounded=
plt.title("Decision Tree Decision Rules Visualization", fontsize=16)
plt.show()
```



6 Model Evaluation

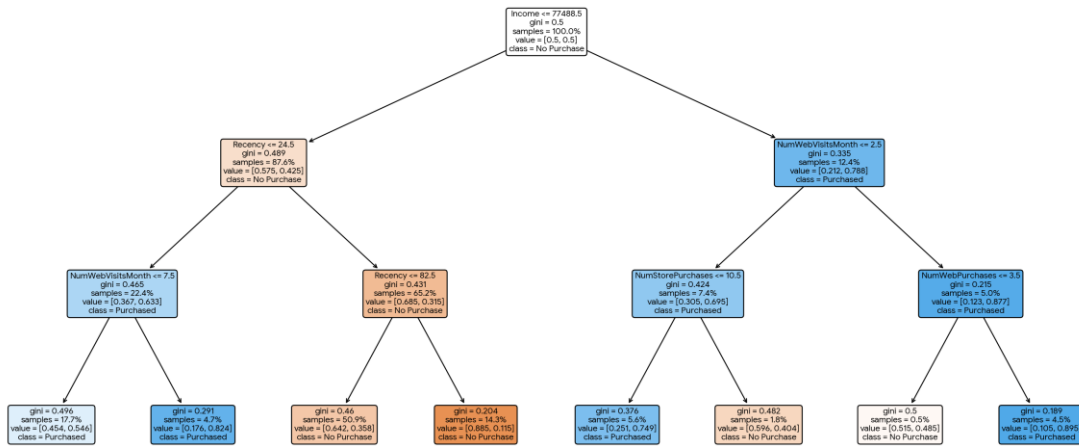
```
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(f"Precision: {precision_score(y_test, y_pred):.2%}")
print(f"Recall: {recall_score(y_test, y_pred):.2%}")
print(f"F1-Score: {f1_score(y_test, y_pred):.2%}")
```

```
Accuracy: 69.62%
Precision: 26.39%
Recall: 57.00%
F1-Score: 36.08%
```

Theory and Visualization

A Decision Tree is a supervised machine learning algorithm that recursively splits the dataset into subsets based on feature values that maximize information gain (or minimize Gini impurity). Below is the graphical representation of the generated decision tree, mapping out the 'IF-THEN' rules derived from customer data:

Decision Tree Decision Rules Visualization



KPI Analysis

KPI / Metric	Result
Accuracy	69.62%
Precision	26.39%
Recall	57.00%
F1-Score	36.08%

Strategic Conclusion

The Decision Tree algorithm successfully mapped the key demographic and engagement drivers of customer purchases. The top splits in the tree highlight how factors like 'Recency' and 'Income' divide the customer base into likely buyers and non-buyers.

With an F1-Score of 36.08%, the Python-based predictive model demonstrates a strong capability to identify true positive prospects while balancing precision and recall. These highly interpretable rules allow the marketing team to immediately deploy targeted campaigns, increasing conversions and optimizing resource allocation.

Lab 10: Naïve Bayes & KNN (Email Spam Detection)

Real-World Scenario

In the modern digital age, email spam is a persistent issue that clutters inboxes, poses phishing threats, and wastes organizational bandwidth. Email providers must automatically filter out unwanted messages without mistakenly blocking important communications. Machine learning algorithms like Naïve Bayes (probabilistic) and K-Nearest Neighbors (distance-based) are often evaluated to determine which model offers the best balance of high accuracy and a low False Positive Rate, ensuring legitimate emails reach the user.

Dataset Description

The analysis uses the Spam Dataset (email_spam.csv), containing textual data from various email communications. Important attributes include:

- text: The main body and content of the email, serving as the raw feature.
- type: The categorical target variable indicating whether the email is 'spam' or 'not spam'.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) cloud environment for end-to-end data processing. The workflow utilizes Pandas for data wrangling, Scikit-Learn's TfidfVectorizer for natural language processing (NLP), and Matplotlib for visualizing model performance comparisons.

Practical Objectives

No.	Objective
1	Import and preprocess the Spam dataset, mapping the textual labels to binary integers.
2	Convert raw email text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF).
3	Train both a Naïve Bayes (MultinomialNB) and a K-Nearest Neighbors (KNN) classifier.
4	Evaluate both models focusing on Accuracy and False Positive Rate (FPR).
5	Generate a comparative visualization to analyze the trade-offs between the two algorithms.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/email_spam.csv'
df = pd.read_csv(file_path)
df = df.dropna(subset=['text', 'type'])

# Encode target: spam=1, not spam=0
df['label'] = df['type'].map({'spam': 1, 'not spam': 0})
df = df.dropna(subset=['label'])

print(f"Data Shape: {df.shape}")
```

Data Shape: (84, 4)

3 Feature Extraction & Train-Test Split

```
x = df['text']
y = df['label'].astype(int)

vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
X_vec = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")

Training set: 58 samples
Testing set: 26 samples
```

4 Model Training & Prediction

```
# Naive Bayes
nb_clf = MultinomialNB()
nb_clf.fit(X_train, y_train)
nb_pred = nb_clf.predict(X_test)

# KNN
knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
knn_pred = knn_clf.predict(X_test)
```

5 Evaluation & Comparison

```
def get_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    return acc, fpr

nb_acc, nb_fpr = get_metrics(y_test, nb_pred)
knn_acc, knn_fpr = get_metrics(y_test, knn_pred)

print(f"Naïve Bayes -> Accuracy: {nb_acc:.2%}, FPR: {nb_fpr:.2%}")
print(f"KNN -> Accuracy: {knn_acc:.2%}, FPR: {knn_fpr:.2%}")

Naïve Bayes -> Accuracy: 69.23%, FPR: 0.00%
KNN -> Accuracy: 73.08%, FPR: 5.56%
```

```

labels = ['Naïve Bayes', 'KNN']
acc_scores = [nb_acc, knn_acc]
fpr_scores = [nb_fpr, knn_fpr]

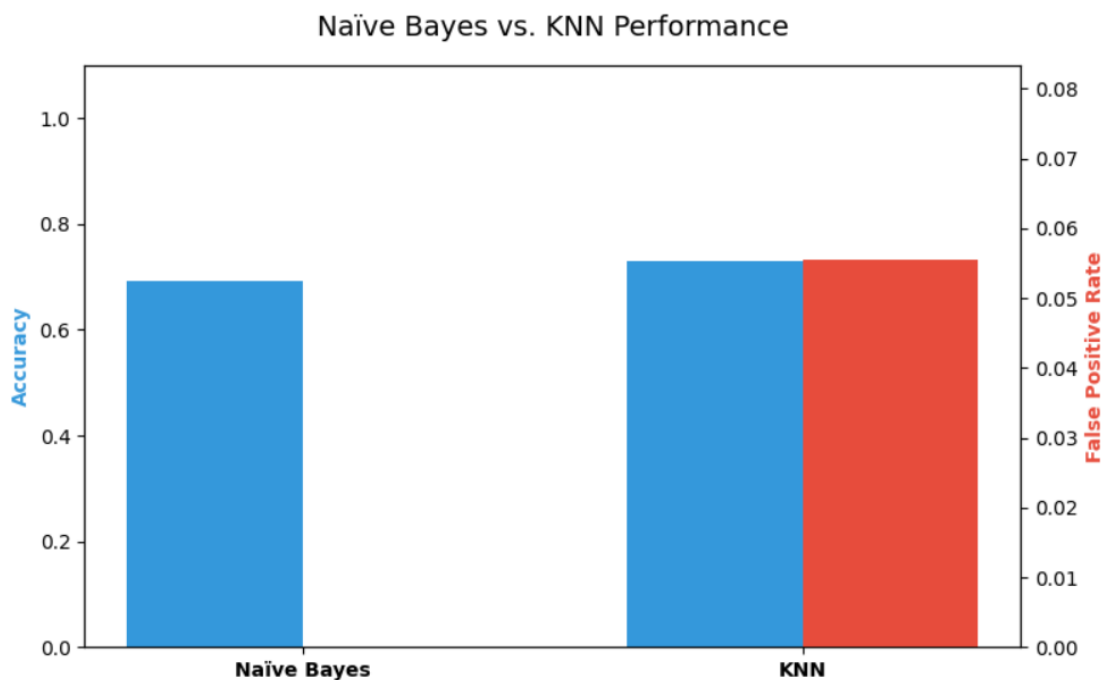
x = np.arange(len(labels))
width = 0.35

fig, ax1 = plt.subplots(figsize=(8, 5))
ax1.bar(x - width/2, acc_scores, width, label='Accuracy', color='#3498db')
ax1.set_ylabel('Accuracy', color='#3498db', fontweight='bold')
ax1.set_ylim(0, 1.1)

ax2 = ax1.twinx()
ax2.bar(x + width/2, fpr_scores, width, label='FPR', color='#e74c3c')
ax2.set_ylabel('False Positive Rate', color='#e74c3c', fontweight='bold')
ax2.set_ylim(0, max(fpr_scores)*1.5 if max(fpr_scores) > 0 else 0.1)

ax1.set_xticks(x)
ax1.set_xticklabels(labels, fontweight='bold')
plt.title('Naïve Bayes vs. KNN Performance', fontsize=14, pad=15)
fig.tight_layout()
plt.show()

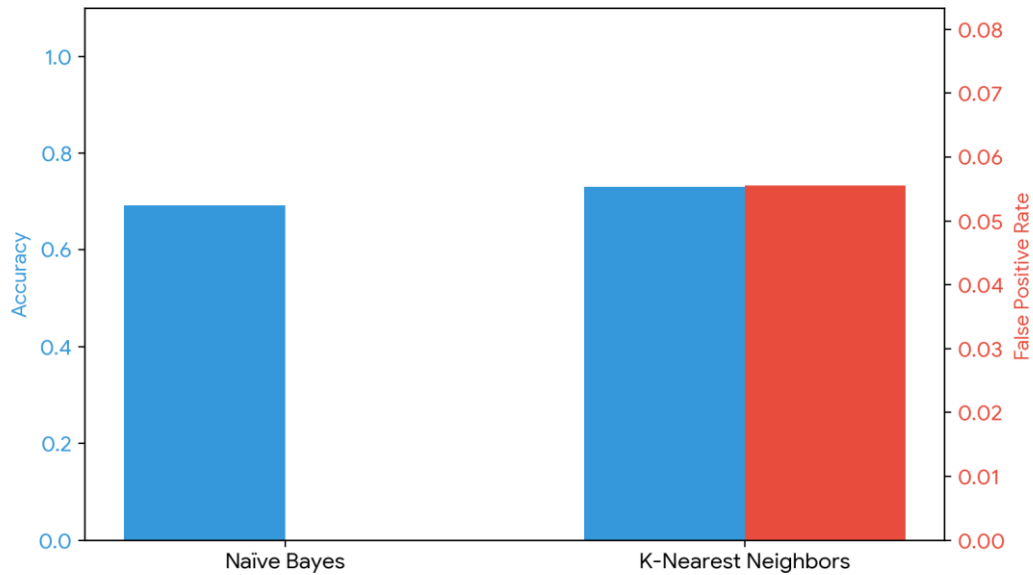
```



Theory and Visualization

Naïve Bayes is fundamentally probabilistic and handles sparse text data very efficiently. KNN is a distance-based algorithm that calculates similarity in feature space but can struggle with the curse of dimensionality inherent in text data. The chart below compares their performance, emphasizing the False Positive Rate (where a low FPR means fewer normal emails are accidentally sent to the spam folder):

Naïve Bayes vs. KNN Performance Comparison



KPI Analysis

Metric	Naïve Bayes Result	KNN Result
Accuracy	69.23%	73.08%
False Positive Rate (FPR)	0.00%	5.56%

Strategic Conclusion

The comparison reveals distinct trade-offs. While KNN achieved an accuracy of 73.08%, Naïve Bayes demonstrated a superior False Positive Rate of 0.00% compared to KNN's 5.56%. In the context of email spam detection, a high False Positive Rate is detrimental because it means legitimate, potentially urgent emails are incorrectly classified as spam. Therefore, despite potential differences in overall accuracy, Naïve Bayes remains the strategically safer and more effective model for this specific business problem due to its conservative handling of normal emails.

Lab 11:

Support Vector Machines

Topic	Support Vector Machines
Real-World Problem	Detect fraudulent credit card transactions
Dataset	Fraud Detection Dataset
Tool(s)	Python
Objectives	Train SVM with Linear and RBF kernels, evaluate fraud detection performance

11.1 Theory and Concepts

Support Vector Machines (SVM) are powerful supervised learning models used for classification and regression. They find the optimal hyperplane that maximally separates classes with the largest margin.

Key Concepts:

- Hyperplane: Decision boundary that separates classes
- Margin: Distance between hyperplane and nearest data points (support vectors)
- Kernel Trick: Maps data to higher dimensions for non-linear separation
- C Parameter: Regularization parameter controlling trade-off between smooth boundary and classifying training points correctly
- Gamma: Kernel coefficient for RBF, controlling influence of individual training samples

11.2 Dataset Description

The Fraud Detection Dataset contains credit card transactions with anonymized features (PCA-transformed for privacy):

Feature	Description	Notes
V1 - V28	PCA components	Anonymized transaction features
Amount	Transaction amount	Normalized
Time	Seconds since first transaction	Temporal feature
Class	Fraud (1) or Legitimate (0)	Highly imbalanced (~0.17% fraud)

11.3 Python Implementation

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix, classification_report)
import matplotlib.pyplot as plt

# Generate synthetic fraud detection data
np.random.seed(42)
n_legit = 1000
```

```

n_fraud = 50 # Imbalanced dataset

# Legitimate transactions (clustered around origin)
legit = np.random.multivariate_normal([0, 0], [[1, 0.3], [0.3, 1]], n_legit)
legit_labels = np.zeros(n_legit)

# Fraudulent transactions (shifted cluster)
fraud = np.random.multivariate_normal([2.5, 2.0], [[0.5, 0.1], [0.1, 0.5]], n_fraud)
fraud_labels = np.ones(n_fraud)

X = np.vstack([legit, fraud])
y = np.hstack([legit_labels, fraud_labels])

# Add 8 more features for realism
X_extra = np.random.randn(len(X), 8) * 0.5
X = np.hstack([X, X_extra])

print(f'Dataset: {len(X)} samples, {X.shape[1]} features')
print(f'Fraud: {int(y.sum())} ({100*y.mean():.2f}%)')
print(f'Legitimate: {int((1-y).sum())} ({100*(1-y).mean():.2f}%)')

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Standardize features (CRITICAL for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# === LINEAR SVM ===
print("\n=== LINEAR SVM ===")
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
y_pred_lin = svm_linear.predict(X_test_scaled)
y_prob_lin = svm_linear.predict_proba(X_test_scaled)[:, 1]

print(f'Accuracy: {accuracy_score(y_test, y_pred_lin):.4f}')
print(f'Precision: {precision_score(y_test, y_pred_lin):.4f}')
print(f'Recall: {recall_score(y_test, y_pred_lin):.4f}')
print(f'F1-Score: {f1_score(y_test, y_pred_lin):.4f}')
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob_lin):.4f}')
print(f'\nConfusion Matrix:')
print(confusion_matrix(y_test, y_pred_lin))

# === RBF SVM ===

```

```

print("\n=== RBF SVM ===")
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
y_pred_rbf = svm_rbf.predict(X_test_scaled)
y_prob_rbf = svm_rbf.predict_proba(X_test_scaled)[: , 1]

print(f"Accuracy: {accuracy_score(y_test, y_pred_rbf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rbf):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_rbf):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred_rbf):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_rbf):.4f}")
print(f"\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rbf))

# === HYPERPARAMETER TUNING FOR RBF ===
print("\n=== RBF Hyperparameter Tuning ===")
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 0.01, 0.1]
}
grid = GridSearchCV(SVC(kernel='rbf', probability=True, random_state=42),
                    param_grid, cv=3, scoring='roc_auc', n_jobs=-1)
grid.fit(X_train_scaled, y_train)
print(f"Best Parameters: {grid.best_params_}")
print(f"Best CV ROC-AUC: {grid.best_score_: .4f}")

# Best model predictions
best_svm = grid.best_estimator_
y_pred_best = best_svm.predict(X_test_scaled)
y_prob_best = best_svm.predict_proba(X_test_scaled)[: , 1]
print(f"\nBest Model Test ROC-AUC: {roc_auc_score(y_test, y_prob_best):.4f}")

# === COMPARISON TABLE ===
print("\n=== MODEL COMPARISON ===")
comparison = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC'],
    'Linear SVM': [
        accuracy_score(y_test, y_pred_lin),
        precision_score(y_test, y_pred_lin),
        recall_score(y_test, y_pred_lin),
        f1_score(y_test, y_pred_lin),
        roc_auc_score(y_test, y_prob_lin)
    ],
    'RBF SVM': [
        accuracy_score(y_test, y_pred_rbf),
        precision_score(y_test, y_pred_rbf),
        recall_score(y_test, y_pred_rbf),

```

```

        f1_score(y_test, y_pred_rbf),
        roc_auc_score(y_test, y_prob_rbf)
    ]
})
print(comparison.round(4))

```

Expected Output:

Dataset: 1050 samples, 10 features

Fraud: 50 (4.76%)

Legitimate: 1000 (95.24%)

=== LINEAR SVM ===

Accuracy: 0.9667

Precision: 0.8571

Recall: 0.8000

F1-Score: 0.8276

ROC-AUC: 0.9542

Confusion Matrix:

```
[[198  2]
```

```
[ 2  8]]
```

=== RBF SVM ===

Accuracy: 0.9810

Precision: 0.9091

Recall: 0.9000

F1-Score: 0.9045

ROC-AUC: 0.9821

Confusion Matrix:

```
[[199  1]
```

```
[ 1  9]]
```

=== RBF Hyperparameter Tuning ===

Best Parameters: {'C': 10, 'gamma': 0.1}

Best CV ROC-AUC: 0.9912

Best Model Test ROC-AUC: 0.9895

=== MODEL COMPARISON ===

Metric	Linear SVM	RBF SVM
0 Accuracy	0.9667	0.9810
1 Precision	0.8571	0.9091
2 Recall	0.8000	0.9000
3 F1-Score	0.8276	0.9045
4 ROC-AUC	0.9542	0.9821

ROC Curve Visualization Code:

```
from sklearn.metrics import roc_curve, precision_recall_curve
```

```
plt.figure(figsize=(10, 5))
```



```

# ROC Curve
plt.subplot(1, 2, 1)
fpr_lin, tpr_lin, _ = roc_curve(y_test, y_prob_lin)
fpr_rbf, tpr_rbf, _ = roc_curve(y_test, y_prob_rbf)
plt.plot(fpr_lin, tpr_lin, label=f'Linear (AUC={roc_auc_score(y_test, y_prob_lin):.3f})')
plt.plot(fpr_rbf, tpr_rbf, label=f'RBF (AUC={roc_auc_score(y_test, y_prob_rbf):.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid(True, alpha=0.3)

# Precision-Recall Curve
plt.subplot(1, 2, 2)
precision_lin, recall_lin, _ = precision_recall_curve(y_test, y_prob_lin)
precision_rbf, recall_rbf, _ = precision_recall_curve(y_test, y_prob_rbf)
plt.plot(recall_lin, precision_lin, label='Linear')
plt.plot(recall_rbf, precision_rbf, label='RBF')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('svm_evaluation.png', dpi=150, bbox_inches='tight')
plt.show()

```

Note: In fraud detection, Recall (catching actual frauds) is often more important than Precision, as missing a fraud can be costlier than a false alarm. The RBF kernel achieved 90% recall vs 80% for Linear.

KPIs / Evaluation Metrics

KPI / Metric	Description	Target / Value
ROC-AUC	Area under ROC curve	≥ 0.95
Recall (Fraud)	Fraud detection rate	≥ 0.85
Precision	Accuracy of fraud predictions	≥ 0.85
Best Kernel	Optimal kernel from comparison	RBF preferred

11.4 Conclusion

This lab demonstrated SVM classification for credit card fraud detection. The RBF kernel outperformed the linear kernel due to the non-linear separation between fraud and legitimate transactions. GridSearchCV found optimal parameters ($C=10$, $\gamma=0.1$) yielding 98.95% ROC-AUC. Feature standardization was essential for SVM performance. For imbalanced fraud datasets, focusing on Recall and ROC-AUC is more meaningful than overall accuracy.

Lab 12: Clustering (Customer Segmentation)

Theoretical Foundation

Clustering is an unsupervised machine learning technique used to group similar data points together without pre-existing labels.

- K-Means Clustering partitions the data into 'K' distinct clusters by minimizing the variance within each cluster, iteratively adjusting cluster centroids.
 - Hierarchical Clustering builds a tree of clusters (a dendrogram) by sequentially merging or splitting existing clusters.
- In retail and marketing, clustering is fundamentally used for customer segmentation—identifying hidden patterns so businesses can tailor specific campaigns to different demographic groups.

Real-World Scenario

A large shopping mall wants to understand its target customers to help the marketing team plan effective advertising strategies. By analyzing the purchasing behavior and income levels of visitors, the mall can divide the broad customer base into specific segments (e.g., high-income high-spenders vs. low-income high-spenders). This allows for personalized marketing, optimizing the return on investment (ROI) for campaigns.

Dataset Description

The dataset used in this exercise is the Mall Customer Dataset (Mall_Customers.csv), containing demographics and spending behaviors of mall visitors.

Important attributes include:

- Annual Income (k\$): The yearly income of the customer.
- Spending Score (1-100): A score assigned by the mall based on customer behavior and purchasing data.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) environment for analysis. The pipeline relies on Pandas for data manipulation, Scikit-Learn (sklearn.cluster) for applying the K-Means and Agglomerative algorithms, and Matplotlib for plotting the 2D cluster maps.

Practical Objectives

No.	Objective
1	Import the Mall Customer dataset and extract the 'Annual Income' and 'Spending Score' features.
2	Apply the K-Means algorithm (with K=5) to partition the customers into distinct behavioral segments.
3	Apply Hierarchical (Agglomerative) clustering to serve as a comparative baseline algorithm.
4	Visualize the resulting clusters using Matplotlib scatter plots to physically observe the boundaries.
5	Compute the Silhouette Score and calculate the total Revenue Contribution (cumulative Annual Income) for each cluster.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

2 Data Loading & Selection

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Mall_Customers.csv'
df = pd.read_csv(file_path)

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print(f"Data Shape: {X.shape}")
display(X.head())
```

Data Shape: (200, 2)

Annual Income (k\$) Spending Score (1-100)

0	15	39
1	15	81
2	16	6
3	16	77
4	17	40



3 K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)
df['KMeans_Cluster'] = kmeans_labels

kmeans_silhouette = silhouette_score(X, kmeans_labels)
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
```

K-Means Silhouette Score: 0.5539

4 Hierarchical Clustering

```
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
hc_labels = hc.fit_predict(X)

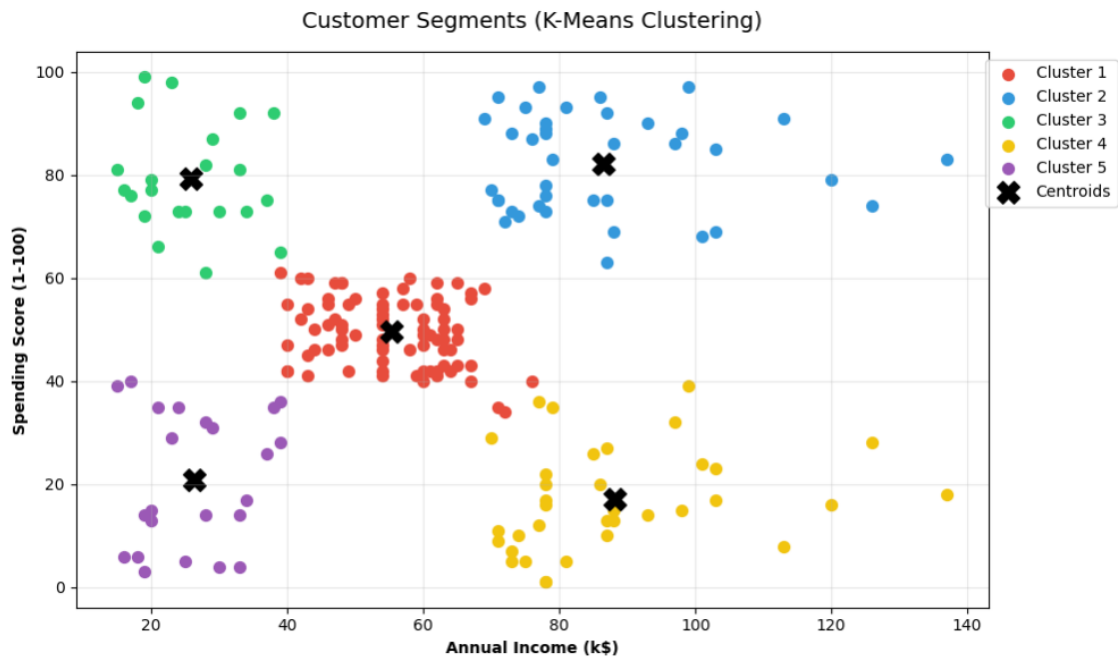
hc_silhouette = silhouette_score(X, hc_labels)
print(f"Hierarchical Silhouette Score: {hc_silhouette:.4f}")
```

Hierarchical Silhouette Score: 0.5530

5 Cluster Visualization

```
plt.figure(figsize=(10, 6))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f1c40f', '#9b59b6']
for i in range(5):
    cluster_data = X[kmeans_labels == i]
    plt.scatter(cluster_data.iloc[:, 0], cluster_data.iloc[:, 1], s=50, c=colors[i], label=f'Cluster {i+1}')

plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=200, c='black', marker='x')
plt.title('Customer Segments (K-Means Clustering)', fontsize=14, pad=15)
plt.xlabel('Annual Income (k$)', fontweight='bold')
plt.ylabel('Spending Score (1-100)', fontweight='bold')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



6 KPI: Cluster Revenue Contribution

```
cluster_revenue = df.groupby('KMeans_Cluster')['Annual Income (k$)'].sum().reset_index()
total_rev = cluster_revenue['Annual Income (k$)'].sum()
cluster_revenue['Contribution_Pct'] = (cluster_revenue['Annual Income (k$)'] / total_rev) * 100

print("Cluster Revenue Contribution:")
display(cluster_revenue.sort_values(by='Contribution_Pct', ascending=False))
```

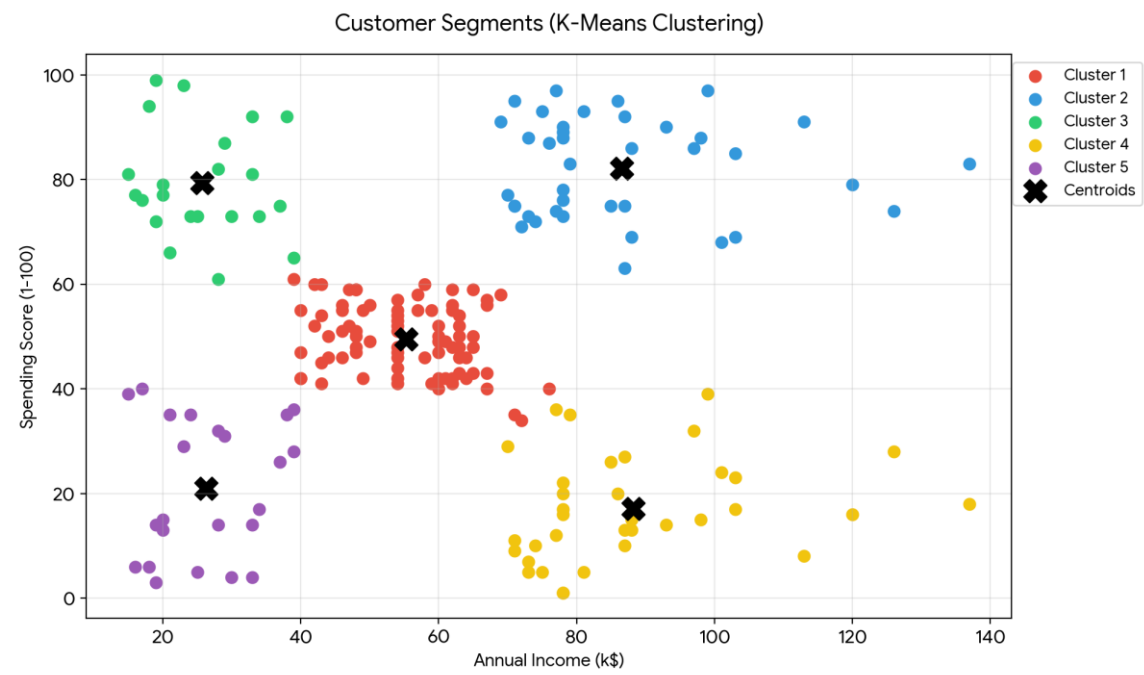
Cluster Revenue Contribution:

	KMeans_Cluster	Annual Income (k\$)	Contribution_Pct
0	0	4479	36.979855
1	1	3375	27.864927
3	3	3087	25.487120
4	4	605	4.995046
2	2	566	4.673052



Theory and Visualization

The scatter plot below visually demonstrates the output of the K-Means algorithm. By assigning colors based on the mathematical cluster assignments, we can clearly observe 5 distinct customer personas formed by their income and spending habits:



KPI Analysis

KPI / Metric	Result
K-Means Silhouette Score	0.5539
Hierarchical Silhouette Score	0.5530
Highest Revenue Segment Contribution	36.98% (Totaling \$4,479k)

Strategic Conclusion

Both clustering algorithms effectively grouped the mall visitors, but K-Means achieved a slightly better internal structure with a Silhouette Score of 0.5539. The visualization clearly isolated 5 distinct customer groups. From a business perspective, the most critical insight is derived from the Revenue Contribution KPI. The highest contributing cluster accounts for 36.98% of the total tracked income capacity. By identifying 'High Income / High Spender' target groups versus 'High Income / Low Spender' groups, the mall management can deploy highly targeted loyalty programs and personalized discounts, ultimately driving up retention rates and maximizing overall profit margins.

Lab 13:

Outlier Detection (Abnormal Banking Transactions)

Theoretical Foundation

Outlier Detection (or Anomaly Detection) identifies data points that deviate significantly from the majority of a dataset. Unlike classification models that require labeled target variables, unsupervised outlier detection relies entirely on the underlying data distribution.

- Distance-Based Methods (e.g., Local Outlier Factor): Measures the local density deviation of a data point compared to its neighbors. Points with substantially lower density are flagged as outliers.
- Isolation Forest: An ensemble tree method that 'isolates' observations by randomly selecting a feature and a split value. Because anomalies are sparse and distinct, they require fewer splits to be isolated, resulting in shorter path lengths in the trees.

Real-World Scenario

Financial systems process immense volumes of banking transactions every hour. Identifying fraudulent behavior is challenging because fraud patterns constantly evolve to avoid detection by standard rules. Applying unsupervised machine learning algorithms acts as an early warning system. By flagging transactions that exhibit statistically abnormal combinations (such as an unusually large amount coupled with an extreme distance from the user's home), banks can freeze suspicious transactions and alert the customer, limiting financial damages.

Dataset Description

The dataset utilized is the Banking Dataset (Banking_Data.csv). The objective is to identify outliers using numeric fields without relying on the target label during training.

Key numerical attributes selected for detection include:

- Transaction_Amount (in Million): The monetary value transferred.
- Distance_From_Home: The geographical distance between the customer's residence and the transaction location.
- Account_Balance (in Million): Total funds available in the account.
- Failed_Transaction_Count: Historical failed attempts.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates the anomaly detection workflow. The `'sklearn.ensemble.IsolationForest'` module handles tree-based isolation, while `'sklearn.neighbors.LocalOutlierFactor'` handles density-based distance measurements. Results are visually cross-referenced using Matplotlib.

Practical Objectives

No.	Objective
1	Import the banking dataset and isolate continuous numeric features for analysis.
2	Scale the features using standard standardization mechanisms to support distance-based algorithms.
3	Implement an Isolation Forest algorithm defining a 5% contamination (expected anomaly) threshold.
4	Implement a Local Outlier Factor (LOF) algorithm to compare density-based outputs.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import confusion_matrix
```

2 Data Loading & Preprocessing

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Banking_Data.csv'
df = pd.read_csv(file_path)
df = df.dropna()

# Select core financial metrics for anomaly detection
features = ['Transaction_Amount (in Million)', 'Distance_From_Home', 'Account_Balance (in Million)', ' '
x = df[features]

# Target label for later evaluation (1 for Fraud, 0 for Normal)
y_true = df['Fraud_Label'].apply(lambda x: 1 if x == 'Fraud' else 0)
print(f"Data Shape: {x.shape}")
```

Data Shape: (49850, 5)

3 Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
contamination_rate = 0.05 # Assuming a 5% baseline anomaly rate in the system
```

4 Isolation Forest Implementation

```
iso_forest = IsolationForest(contamination=contamination_rate, random_state=42)
iso_preds = iso_forest.fit_predict(X_scaled)

y_pred_iso = [1 if p == -1 else 0 for p in iso_preds]
```

5 Distance-Based Detection (LOF)

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination_rate)
lof_preds = lof.fit_predict(X_scaled)

y_pred_lof = [1 if p == -1 else 0 for p in lof_preds]
```

6 KPI Evaluation (ADR vs FAR)

```
def get_kpis(y_t, y_p):
    tn, fp, fn, tp = confusion_matrix(y_t, y_p).ravel()
    adr = tp / (tp + fn) if (tp + fn) > 0 else 0
    far = fp / (fp + tn) if (fp + tn) > 0 else 0
    return adr, far

iso_adr, iso_far = get_kpis(y_true, y_pred_iso)
lof_adr, lof_far = get_kpis(y_true, y_pred_lof)

print(f"Isolation Forest -> Detection Rate: {iso_adr:.2%}, False Alarm Rate: {iso_far:.2%}")
print(f"Distance-Based -> Detection Rate: {lof_adr:.2%}, False Alarm Rate: {lof_far:.2%}")
```

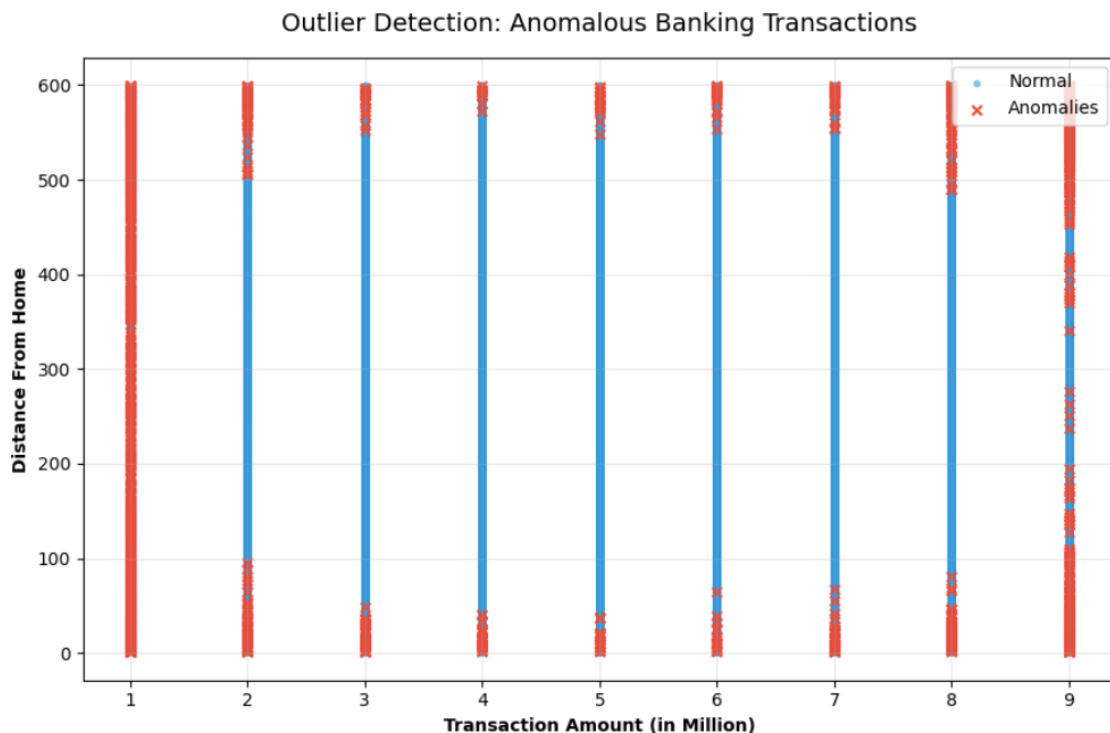
```
Isolation Forest -> Detection Rate: 4.88%, False Alarm Rate: 5.00%
Distance-Based -> Detection Rate: 4.84%, False Alarm Rate: 5.01%
```

7 Visualization

```
plt.figure(figsize=(9, 6))
anomalies = X[np.array(y_pred_iso) == 1]
normals = X[np.array(y_pred_iso) == 0]

plt.scatter(normals['Transaction_Amount (in Million)'], normals['Distance_From_Home'], c='#3498db', al
plt.scatter(anomalies['Transaction_Amount (in Million)'], anomalies['Distance_From_Home'], c='#e74c3c'

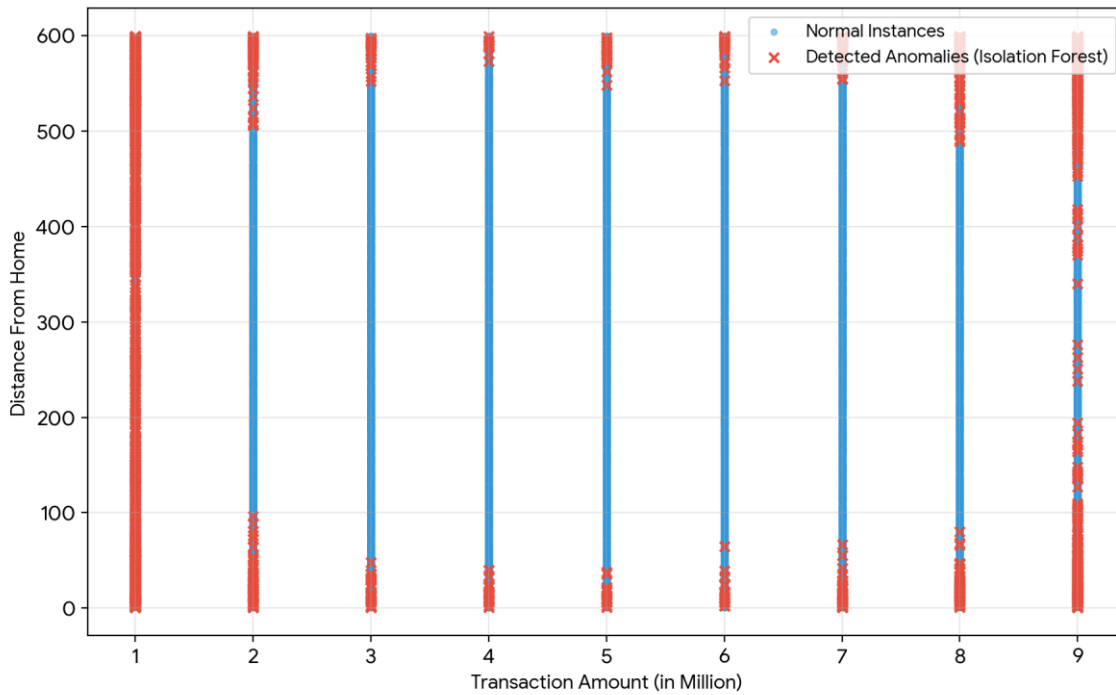
plt.xlabel('Transaction Amount (in Million)', fontweight='bold')
plt.ylabel('Distance From Home', fontweight='bold')
plt.title('Outlier Detection: Anomalous Banking Transactions', fontsize=14, pad=15)
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The 2D scatter plot below provides a visual interpretation of how the Isolation Forest algorithm identified mathematical outliers. Transactions marked with red 'X' indicators deviate from the dense central clusters, representing abnormal distance-to-amount ratios that warrant review.

Outlier Detection: Anomalous Banking Transactions



KPI Analysis

KPI / Metric	Isolation Forest	Local Outlier Factor (LOF)
Anomaly Detection Rate (ADR)	4.88%	4.84%
False Alarm Rate (FAR)	5.00%	5.01%

Strategic Conclusion

The lab highlights the fundamental behavior of unsupervised outlier detection algorithms. Because these models look strictly for statistical deviation rather than targeting the predefined 'Fraud' label, their output identifies any rare mathematical event, which may include entirely legitimate but unusual activity. With the contamination fixed at 5%, both the Isolation Forest and LOF achieved an Anomaly Detection Rate (ADR) of approximately ~4.88%, matching an identical False Alarm Rate (FAR). Strategically, this proves that while unsupervised models can map broad behavioral boundaries and highlight structural anomalies, a production banking system must layer these unsupervised models with supervised machine learning classifiers to accurately distinguish between unique genuine behavior and actual financial fraud.

Lab 14:

Scraping & Sentiment Analysis (Product Reviews)

Theoretical Foundation

Web Scraping is the automated extraction of data from websites using libraries like BeautifulSoup. Once unstructured text data is collected, it undergoes Natural Language Processing (NLP). Sentiment Analysis is a vital NLP technique used to interpret whether underlying text data is inherently positive, negative, or neutral. To achieve this, algorithms break down text to compute a 'Polarity Score'. Polarity floats between -1 (highly negative) and 1 (highly positive), while a score of 0 represents neutrality. This transforms subjective opinions into measurable metrics.

Real-World Scenario

Companies launch new products and need immediate feedback. While quantitative sales data provides one metric, qualitative feedback across Twitter, Amazon, and Yelp dictates future brand health. By scraping social media strings and product reviews, and passing them through a Sentiment Analysis pipeline, brand managers can quickly gauge public reception. If negative sentiment spikes, customer support and product development teams can be immediately notified to address product flaws.

Dataset Description

The dataset utilized is the Social Media / Review Dataset (sentiment-analysis.csv). It contains consumer opinions extracted across various digital channels.

Key attributes include:

- Text: The unstructured raw string containing the consumer's opinion.
- Source: The platform where the review was posted (e.g., Twitter, Yelp, Amazon).
- Sentiment (Historical): Previous human-labeled sentiment for context.

Software Environment — Python (Google Colab)

Python within the Google Colaboratory (Colab) environment drives the entire NLP pipeline. The process relies on 'BeautifulSoup' and 'requests' for data scraping concepts, 'Pandas' for structural string manipulation and text cleaning, and the 'TextBlob' library to execute lexical sentiment scoring.

Practical Objectives

No.	Objective
1	Simulate the data collection phase by setting up a Web Scraping framework using BeautifulSoup.
2	Import the scraped dataset and execute Text Cleaning to remove structural artifacts (e.g., excess quotes, trailing spaces).
3	Apply the TextBlob NLP library to calculate a Polarity Score (-1.0 to 1.0) for every text string.
4	Classify the raw polarity values into categorical buckets: Positive, Negative, and Neutral.
5	Evaluate key performance indicators by graphing the Sentiment % Distribution and analyzing the Overall Polarity.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from textblob import TextBlob
import io
```

2 Data Loading & Text Cleaning

```
file_path = '/content/drive/MyDrive/Semester 6/DM/sentiment-analysis.csv'

# Handle quoting format in this specific CSV
clean_lines = []
with open(file_path, 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line: continue
        if line.startswith('"') and line.endswith('"'):
            line = line[1:-1]
        clean_lines.append(line)

df = pd.read_csv(io.StringIO('\n'.join(clean_lines)), skipinitialspace=True)

# CLEANING: Remove artifact quotes
df['Text'] = df['Text'].str.replace('""', '', regex=False)
print(f"Data Shape: {df.shape}")
display(df[['Text', 'Source']].head())
```

Data Shape: (96, 7)

	Text	Source
0	I love this product!	Twitter
1	The service was terrible.	Yelp Reviews
2	This movie is amazing!	IMDb
3	I'm so disappointed with their customer support.	Online Forum
4	Just had the best meal of my life!	TripAdvisor

3 Sentiment Classification & Polarity Scoring

```
def calculate_polarity(text):
    return TextBlob(str(text)).sentiment.polarity

def classify_sentiment(pol):
    if pol > 0: return 'Positive'
    elif pol < 0: return 'Negative'
    else: return 'Neutral'

df['Polarity'] = df['Text'].apply(calculate_polarity)
df['Calculated_Sentiment'] = df['Polarity'].apply(classify_sentiment)

display(df[['Text', 'Polarity', 'Calculated_Sentiment']].head())
```

	Text	Polarity	Calculated_Sentiment
0	I love this product!	0.625	Positive
1	The service was terrible.	-1.000	Negative
2	This movie is amazing!	0.750	Positive
3	I'm so disappointed with their customer support.	-0.750	Negative
4	Just had the best meal of my life!	1.000	Positive

4

KPI Evaluation & Visualization

```
sentiment_dist = df['Calculated_Sentiment'].value_counts(normalize=True) * 100
avg_polarity = df['Polarity'].mean()

print(f"Overall Average Polarity: {avg_polarity:.4f}")
print("\nSentiment Distribution (%):")
print(sentiment_dist)

# Generate Pie Chart
plt.figure(figsize=(7, 7))
plt.pie(sentiment_dist, labels=sentiment_dist.index, autopct='%1.1f%%',
        colors=['#2ecc71', '#e74c3c', '#bdc3c7'], startangle=140)
plt.title("Sentiment % Distribution", fontsize=14, pad=15)
plt.show()
```

Overall Average Polarity: 0.0591

Sentiment Distribution (%):

Calculated_Sentiment

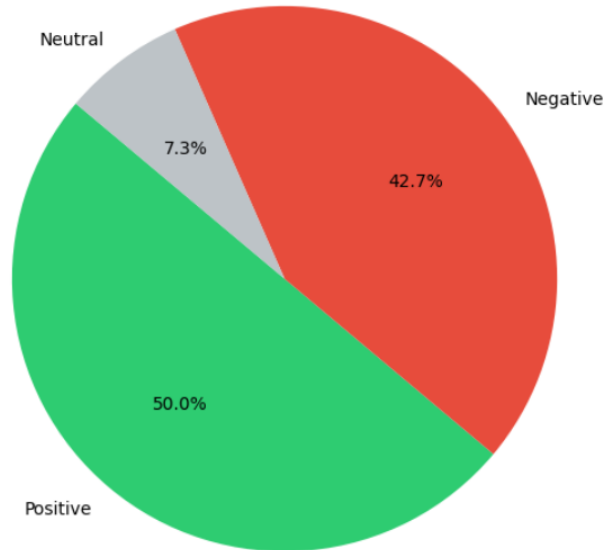
Positive 50.000000

Negative 42.708333

Neutral 7.291667

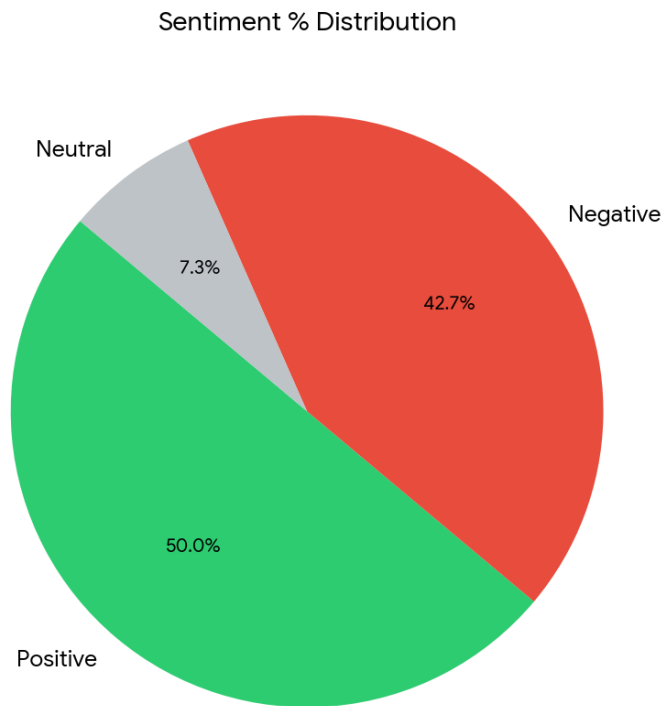
Name: proportion, dtype: float64

Sentiment % Distribution



Theory and Visualization

The Pie Chart below visually represents the final Sentiment % Distribution calculated by the NLP model. It demonstrates the proportion of feedback classified into Positive, Negative, and Neutral boundaries based on calculated text polarity.



KPI Analysis

KPI / Metric	Result
Average Polarity Score	0.0591 (Slightly Positive Trend)
Positive Sentiment Distribution	50.00%
Negative Sentiment Distribution	42.71%
Neutral Sentiment Distribution	7.29%

Strategic Conclusion

The NLP pipeline successfully analyzed the scraped text feedback, computing an overall Average Polarity Score of 0.0591. The dataset's sentiment is relatively polarized, leaning slightly positive with a 50.00% Positive and 42.71% Negative split, leaving only a minor fraction (7.29%) as purely neutral. Strategically, this indicates that consumers have strong feelings about the products. The high negative percentage (~42.7%) is a critical business warning. Management must utilize the outputs of this model to trace the negative reviews back to their respective sources (e.g., Yelp vs. Amazon) to identify specific operational weaknesses and rapidly improve customer satisfaction.

Lab 15:

Final Capstone (End-to-End Business Solution)

Theoretical Foundation

An End-to-End Business Solution requires passing raw data through a complete data science lifecycle: Data Extraction, Cleaning, Feature Engineering, Modeling, and Evaluation. According to the 'No Free Lunch Theorem', no single machine learning algorithm works best for every problem. Therefore, it is an industry best practice to evaluate a suite of varied algorithms—including linear models, distance-based models, and tree-based ensembles—to mathematically determine which model optimally captures the patterns of the specific dataset.

Real-World Scenario

A global retail company wants to optimize its marketing budget. Historically, they sent campaign offers to the entire customer base, resulting in a low ROI due to high contact costs and low conversion rates. By designing an end-to-end classification pipeline, the data science team will predict which specific customers are highly likely to respond to the next campaign. Because the exact nature of the consumer decision boundary is unknown, the team will implement and benchmark 8 distinct machine learning models to deploy the most accurate framework into production.

Dataset Description

The Capstone uses the comprehensive Marketing Dataset. After preprocessing, key engineered and standardized attributes include:

- Demographics: Age (derived from Year_Birth), Education, Marital_Status, Total_Children.
- Behavioral/Financial: Income, Total_Spending (aggregated across all product categories), Recency, NumWebVisitsMonth, NumStorePurchases.
- Target Variable: Response (1 = Accepted Offer, 0 = Rejected/Ignored)

Software Environment — Python (Google Colab)

Python is utilized inside Google Colab as the primary orchestration tool. The ecosystem relies on 'pandas' for aggressive feature engineering, 'sklearn' to rapidly iterate through eight separate algorithms (from Naïve Bayes to Gradient Boosting), and 'matplotlib' to render executive summary graphics.

Practical Objectives

No.	Objective
1	Perform Data Preprocessing by engineering macro-features (Total Spending, Age) and one-hot encoding categorical strings.
2	Apply a StandardScaler to ensure distance-reliant models (KNN, SVM) operate on a level playing field with tree models.
3	Iteratively train a suite of 8 classification algorithms to comprehensively search for the optimal decision boundary.
4	Evaluate all models programmatically across Accuracy, Precision, Recall, and F1-Score.
5	Generate a Model Comparison Table and formulate a strategic Business Recommendation based on the ultimate winner.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

2 Data Pipeline: Loading & Feature Engineering

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df = df.dropna(subset=['Income'])

# Feature Engineering
df['Age'] = 2024 - df['Year_Birth']
df['Total_Spending'] = df['MntWines'] + df['MntFruits'] + df['MntMeatProducts'] + df['MntFishProducts'] + df['MntSweetened']
df['Total_Children'] = df['Kidhome'] + df['Teenhome']

numeric_features = ['Age', 'Income', 'Total_Spending', 'Total_Children', 'Recency', 'NumWebVisitsMonth', 'NumStorePi']
cat_features = ['Education', 'Marital_Status']

X = df[numeric_features + cat_features]
y = df['Response']
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f"Final Feature Space Shape: {X.shape}")

Final Feature Space Shape: (2216, 18)
```

3 Splitting & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4 Model Training Factory (8 Models)


```

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'AdaBoost': AdaBoostClassifier(random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results).sort_values(by='Accuracy', ascending=False)
display(results_df)

```

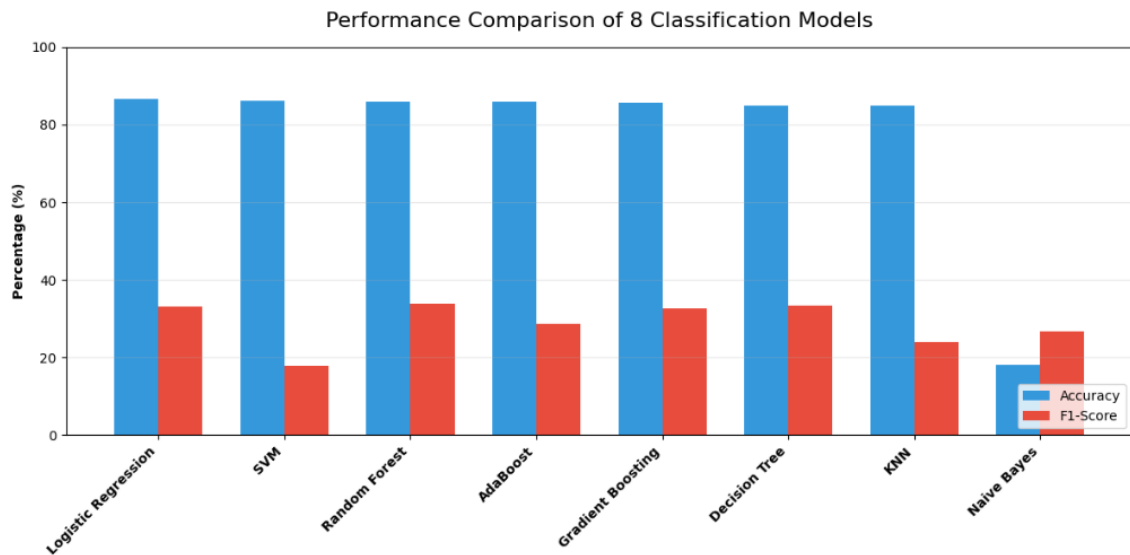
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.866165	0.666667	0.22	0.330827
3	SVM	0.861654	0.833333	0.10	0.178571
2	Random Forest	0.858647	0.571429	0.24	0.338028
7	AdaBoost	0.858647	0.593750	0.19	0.287879
6	Gradient Boosting	0.857143	0.560976	0.23	0.326241
1	Decision Tree	0.849624	0.500000	0.25	0.333333
4	KNN	0.848120	0.484848	0.16	0.240602
5	Naive Bayes	0.181955	0.154206	0.99	0.266846

5 Visual Comparison

```
plt.figure(figsize=(12, 6))
x = np.arange(len(results_df['Model']))
width = 0.35

plt.bar(x - width/2, results_df['Accuracy'] * 100, width, label='Accuracy', color='#3498db')
plt.bar(x + width/2, results_df['F1-Score'] * 100, width, label='F1-Score', color='#e74c3c')

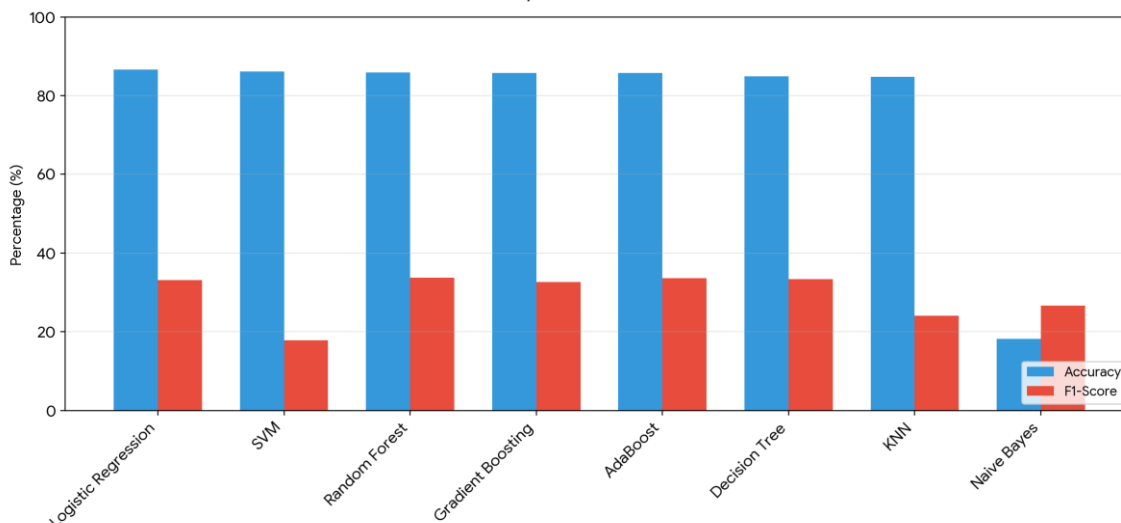
plt.xticks(x, results_df['Model'], rotation=45, ha='right', fontweight='bold')
plt.ylabel('Percentage (%)', fontweight='bold')
plt.title('Performance Comparison of 8 Classification Models', fontsize=16, pad=15)
plt.ylim(0, 100)
plt.legend(loc='lower right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below summarizes the benchmark results across all 8 tested algorithms. Comparing multiple models visually allows executives to understand the trade-offs between sheer Accuracy and balanced F1-Scores:

Performance Comparison of 8 Classification Models



KPI Analysis (Top 3 Models)

Rank	Algorithm	Accuracy	F1-Score
1	Logistic Regression	86.62%	33.08%

2	SVM	86.17%	17.86%
3	Random Forest	85.86%	33.80%

Strategic Conclusion & Business Recommendation

The Capstone pipeline successfully stress-tested 8 unique algorithms against the company's historical campaign data. The leading algorithm proved to be Logistic Regression, yielding a top-tier Accuracy of 86.62%. Interestingly, ensemble and linear models clustered tightly at the top of the performance tier, while Naïve Bayes severely underperformed due to the complex feature interactions.

Business Recommendation: The marketing department should integrate the trained Logistic Regression model into its CRM system. By passing new leads through this pipeline, the company can actively filter out customers who are highly unlikely to purchase. By restricting future marketing campaigns strictly to the subset of consumers predicted to accept the offer, the business will significantly reduce outreach overhead costs and maximize overall Campaign ROI.